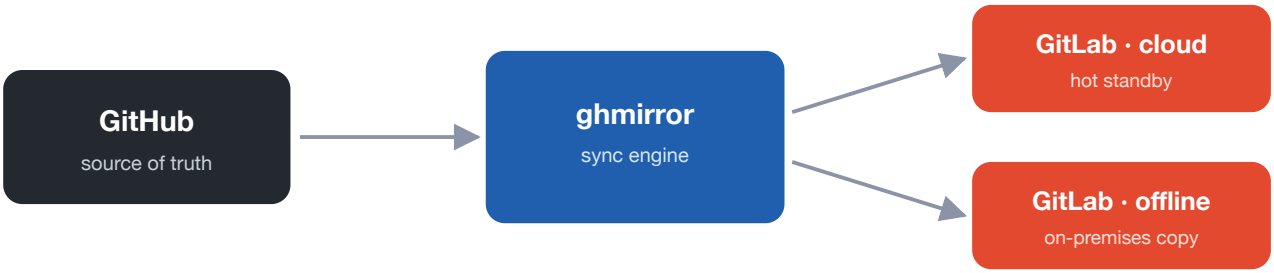
GitHub → GitLab Mirror & Failover

How the system works, how to set it up, how to operate it day to day, and how to switch the team to GitLab if GitHub is lost.



Document	**Architecture, Setup and Operations Guide**
Version	**1.1 — engine v0.1**
Date	**4 November 2026**
Source	**github.com/shoubhiksaha/ghmirror (private)**
Tested on	**GitLab CE 19.5.3 (two instances, a from-scratch setup, a recorded failover drill) · GitHub.com · Python 3.12**

Contents

1 Executive summary 3

2 What is mirrored 4

3 How it works 5

3.1 Components 5

3.2 Three layers of copy 6

3.3 Mirror accounts 6

3.4 Same numbers, same diffs 6

3.5 The side channel 7

3.6 The sync pipeline 7

3.7 Safety mechanisms 8

4 Operating modes and failover 9

5 Setup guide 11

6 A tour of the mirror (screenshots) 16

7 Operations runbook 26

8 Verification and test results 29

9 Limits and known gaps 32

10 Troubleshooting 33

11 Security architecture and threat model 34

12 Capacity, API limits and recovery objectives 36

13 Supported platforms and offline-mode limits 38

14 Recorded failover drill and parity report 39

15 Commercial deployment and support (draft) 42

A Command reference 43

B Configuration reference 44

C Repository layout 46

1 Executive summary

ghmirror keeps a complete, continuously updated copy of a GitHub organization in two self-managed GitLab instances: one in the cloud and one on-premises. It copies code and also the collaboration history around it: issues, pull requests, reviews, comments, releases, people and teams.

If GitHub becomes unavailable, for example because the account is suspended, the organization is deleted or a region has an outage, one command (`ghmirror promote`) turns a GitLab copy into the team's primary code host. People keep their usernames, their history and their SSH keys. The only real change for a developer is the remote URL. The other GitLab then follows the new primary automatically.

2
independent GitLab copies (cloud + offline)

1:1
issue and PR numbers, authors and timestamps

0
drift in the parity check after every test phase

1 cmd
to fail over: `ghmirror promote`

Key properties

Property	How it is achieved
Faithful	Every issue, PR, comment and review keeps its number, original author and original timestamps. Merged PRs show the exact diff that was reviewed. Line comments sit on the same lines.
Automatic	GitHub webhooks trigger a sync within seconds. Scheduled passes catch anything a webhook missed. A nightly parity check compares both sides and repairs differences.
Private-repo safe	Uses a GitHub App or token with read access to private repositories. All copies stay *private* on GitLab (`visibility_cap`).
Never destructive	Nothing that disappears from GitHub is deleted from GitLab. A deleted repo is *archived*, and a deleted issue is closed and labelled. A sudden mass disappearance halts all writes.
Reversible failover	`promote` records each change as it makes it, and `demote` undoes it exactly. That lets the team run monthly failover drills safely.
Quick to set up	One command on the engine host (`./setup.sh`) asks a few questions, creates the GitHub App, checks every connection and starts mirroring. Each GitLab host is prepared by one line that setup prints. No manual GitLab or GitHub configuration (section 5).
Independent of GitHub	Besides the two GitLab copies, the engine keeps bare git mirrors and a raw JSON archive of all metadata on its own disk. A lost GitLab is rebuilt from the other one, and the code alone can be pushed back from the bare mirrors (section 7.5).

> **Status**
>
> The engine is built and tested end to end against a real GitHub account and two GitLab CE 19.5.3 instances. The final run passed every check, including a full failover drill (section 8). Organization teams, GitHub App authentication, wikis and LFS are implemented but were not exercised end to end, because the test account is a personal account (section 9). The one-command setup was tested from scratch over real SSH (section 8.3).

2 What is mirrored

The table maps every GitHub object to its GitLab counterpart. "Exact" means the visible fields match: number, author, timestamps, state, text, labels and position.

GitHub	GitLab	Fidelity	Notes
Repository, settings, topics	Project in the mirror group	exact	Description, default branch, topics, archived flag; renames follow.
Branches, tags, commits	Same refs (ref-exact push)	exact	Force-pushes are mirrored. Overwritten tips are kept under `refs/ghmirror-backup/`.
Issues	Issues, same number	exact	Author, dates, state, closer, labels, milestone, assignees, lock.
Pull requests	Merge requests, same number	exact	Open, closed or merged, with merger and dates. Drafts and reviewers. The exact reviewed diff, even for squash and rebase merges.
Comments, edits, deletions	Notes	exact	Original author and time. Edits update the note and deletions remove it.
Reviews	Review notes + approvals	exact	"Changes requested" is marked in the note. Approvals carry the original date.
Line comments / threads	Positioned diff discussions	exact	Single-line and multi-line, replies and resolution. Falls back to a quoted note if a line no longer exists.
Reactions	Award emoji	exact	On issues, PRs and comments.
Labels, milestones	Labels, milestones	exact	Colour, description, due date, state.
Releases + asset files	Releases + generic package files	exact	Assets are re-hosted on GitLab, so links do not point back at GitHub.
Attachments in text	GitLab uploads	exact	Images and files pasted into issues and comments are copied and links rewritten.
Users	Mirror accounts	exact	Same username, name, avatar, SSH and GPG keys. Bots become `<name>-bot`.
Members, teams, collaborators	Group members, subgroups, project shares	built	Applied at failover. Not exercised end to end (no organization account in the tests).
Branch protection	Protected branches	at failover	Archived continuously and applied by `promote`. While mirroring, only the bot may push.
Wikis, LFS objects	Project wiki, LFS	built	Implemented but not exercised (GitHub wikis cannot be created by API).
Discussions, Projects, Actions history	—	not copied	No GitLab equivalent, and not captured in the L2 archive either.
Secrets, deploy-key private halves	—	never	GitHub never exposes them. Re-create them in GitLab CI settings.

3 How it works

3.1 Components

ghmirror is one Python service (the *engine*), packaged as a Docker image, with a PostgreSQL database for its state. It reads from GitHub over the REST and GraphQL APIs and git. It writes to each GitLab through the REST API, git pushes, and a small *side channel* for the few fields the API cannot set.

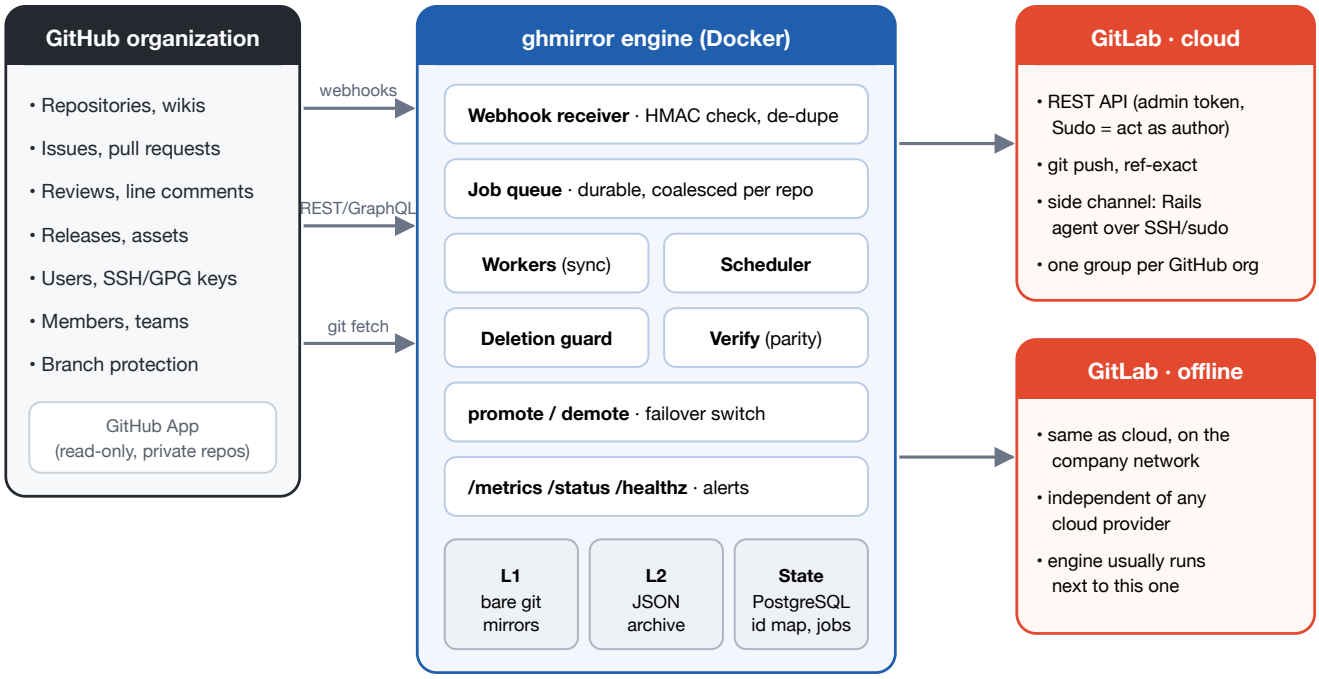


Figure 1. System components. The engine only reads from GitHub and writes the same changes to every GitLab target.

Component	Responsibility	Code
Webhook receiver	Verifies GitHub's HMAC signature, drops duplicate deliveries and turns each event into a job (git, metadata, full, repos, org).	`server.py`
Job queue	Durable (survives restarts), coalesces repeated jobs for the same repo, keeps per-repo order, retries with back-off and moves failing jobs to a dead-letter state.	`db.py`
Workers	Run a repo sync against every target: git first, then metadata.	`sync.py`, `gitmirror.py`
Scheduler	Periodic passes (repos hourly, metadata hourly, full nightly, git 6-hourly, verify nightly, migration archive weekly). Covers missed webhooks.	`server.py`
Identity manager	Creates and maintains a mirror account for every GitHub user who appears in the data.	`identity.py`
Deletion guard	Detects disappearing data and halts or archives instead of deleting.	`engine.py`
Side channel	Persistent `gitlab-rails` agent for fields the API cannot set (section 3.5).	`sidechannel.py/.rb`

Verify	Compares GitHub and each GitLab field by field and reports drift (optionally repairs it).	`verify.py`
Promote / demote	Failover switch (section 4).	`promote.py`
Redirector	Turns old `github.com/org/repo/pull/12` links into the GitLab URL of `!12`.	`redirector.py`

3.2 Three layers of copy

The design keeps three independent copies of the organization. Each layer protects against a different failure.

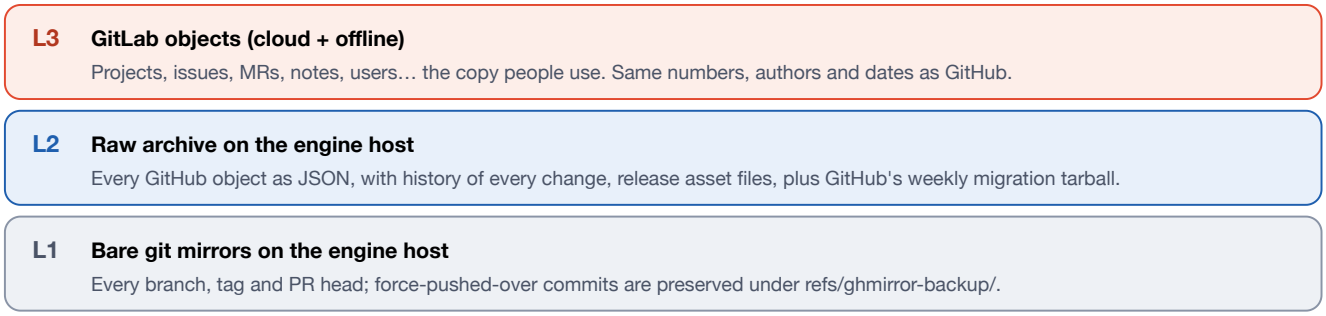


Figure 2. L1 and L2 live on the engine host and do not depend on GitLab. The code of either GitLab can be rebuilt from L1. L2 preserves the record as JSON, but replaying it into a new GitLab is not automated (§7.5, §9).

3.3 Mirror accounts (identity)

For every GitHub user who appears in the data, the engine creates a GitLab account with the **same username**, display name, avatar, SSH keys and GPG keys. Accounts are matched by a hidden marker (`ghmirror github_id:<id>`) in the admin note, so renames and re-installs never create duplicates.

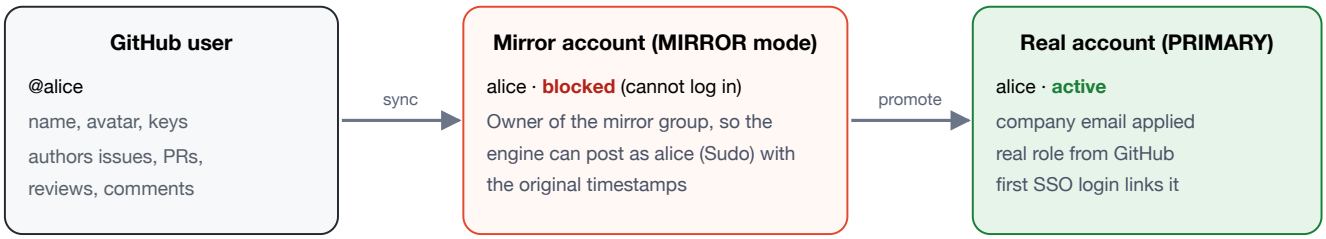


Figure 3. Lifecycle of an account. Bots and people who have left the organization stay blocked at promotion.

While mirroring, nobody can log in as a mirror account and nothing can be pushed by hand. The copy stays an exact image of GitHub. At failover, `promote` activates the human accounts, applies their company e-mail from `identities.yaml` and gives each person the role they had on GitHub. This is the only change needed to "turn mirror accounts into real accounts".

3.4 Same numbers, same diffs

Numbers. GitHub numbers issues and PRs in one shared sequence (#1, #2, #3…). GitLab has separate counters for issues and merge requests. The engine creates objects strictly in number order and keeps them aligned: issue #2 stays issue #2, and PR #3 becomes merge request **!3**, not !1. If anything ever breaks the alignment (for example a failed create), the side channel renumbers the MR and its git ref, and later creations wait until it is fixed.

Diffs. Recreating a merged PR naively against `main` would show an empty diff, because main already contains the change. The engine instead pushes the PR's head and base commits to hidden refs, opens the MR between them, so GitLab stores the exact reviewed diff, then sets the real target branch, state, merger and dates. Line comments are placed on the same file, line and side as on GitHub, including multi-line ranges.

3.5 The side channel

The GitLab API lets an admin create objects *as* another user and backdate most of them. A few fields are read-only in the API: MR creation date, merged and closed state with merger and time, approval dates, release author, and the MR number itself. For these, the engine runs a small Ruby agent inside GitLab's own Rails environment (`gitlab-rails runner`), reached by the configured command prefix (typically `ssh gitlab-host sudo`).

- One agent per engine process. Rails boots once, and operations then stream as JSON lines in batches.
- Every operation is idempotent and written to the audit log.
- Each agent has a unique token. On exit or timeout it is killed on the GitLab host (`pkill -9`), and it exits by itself after 15 minutes without input, so it cannot leak memory on the server.
- Without a side channel the engine still runs. Those few fields then fall back to "now", and the parity check reports them.

3.6 The sync pipeline



Every step is idempotent: a pass that changes nothing on GitHub makes no write on GitLab, and an interrupted pass resumes where it stopped.

Figure 4. One repository sync. Git always goes first, so commits referenced by PRs and releases already exist on GitLab.

1. **Event.** A GitHub webhook (push, issues, pull_request, review, comment, release, member, repository…) or a scheduled pass creates a job.
2. **Queue.** Jobs are stored durably. Repeated events for one repo collapse into one job, and two workers never touch the same repo at once.
3. **Git.** The bare mirror is fetched (branches, tags, PR heads) and pushed ref-exactly to every target. Branch deletions use the API because the mirror's protection rule blocks deletion by push.
4. **Metadata.** Labels, milestones, then every issue and PR in number order with comments, reactions, reviews and line comments, then releases and assets. Each object is matched through the id map. If the map is lost, existing GitLab objects are *adopted* by number, name or author, time and body instead of being created twice.
5. **Side channel.** The fields from 3.5 are applied in one batch per repo.
6. **Verify.** A nightly field-by-field comparison in both directions. Drift raises an alert, and `--repair` re-queues the affected objects.

3.7 Safety mechanisms

Risk	Mechanism
GitHub org deleted, suspended or hacked	**Circuit breaker.** If the owner returns 404, or more objects vanish in one pass than allowed (default: over 2% and over 1 item), all writes stop immediately with an alert. The mirrors keep the last good state.
A repo or issue deleted on GitHub	The repo is **archived** on GitLab, never deleted. The issue is closed and labelled `deleted-on-github`.
History rewritten (force-push)	The old tip is saved under `refs/ghmirror-backup/<time>/…` before the mirror follows.
Slow or overloaded GitLab	Retries with back-off on 429/5xx. A create whose outcome is unknown (timeout, 502) is **looked up before it is resent**, so no duplicate issues, notes, MRs or accounts are made.
Engine crash mid-way	Jobs have leases and are retried. All steps are idempotent. A crashed `promote` can always be undone by `demote`.
Silent divergence	Nightly `verify`: refs, numbers, titles, authors, dates, states, merger, labels, note counts, milestones, releases, and objects that exist only on GitLab.
Someone edits the mirror by hand	Mirror accounts are blocked and only the bot may push (protected `*` rule). Verify reports anything else.

4 Operating modes and failover

Each GitLab target is always in one of three modes. The engine stores the mode, and it changes only through the commands shown.

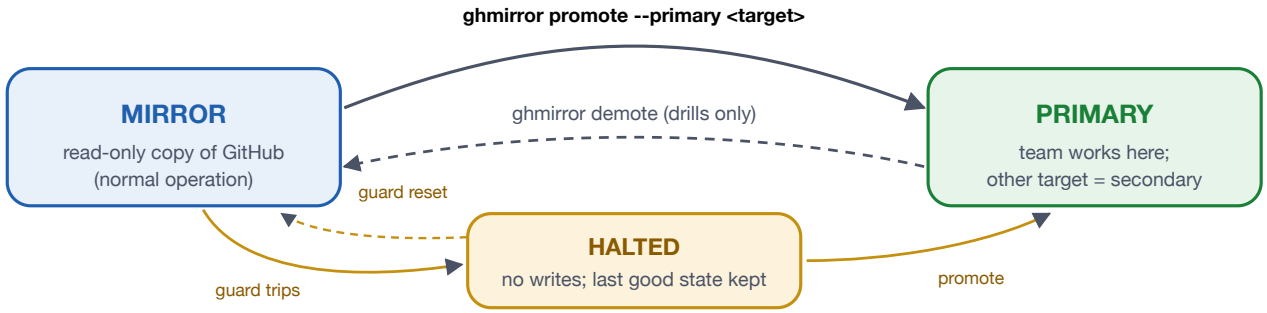


Figure 5. Modes. A guard halt is never cleared automatically: a person decides between `guard reset` (false alarm) and `promote` (GitHub really is gone).

4.1 What `promote` does

1. **Records the promotion** before changing anything, and updates the record after every step. A crash part-way can always be undone.
2. **Stops the GitHub source**, so the engine writes nothing more from GitHub.
3. **Restores real branch protection**, replacing the bot-only rule with the rules archived from GitHub (required reviews, who may push, force-push).
4. **Enables CI/CD** on every project (it is off while mirroring).
5. **Turns mirror accounts into real ones.** Humans are unblocked and get their company email. Bots and former members stay blocked. The temporary Owner role is removed, and real roles are applied: org admins become Owners, members get the base permission, teams become subgroups with project shares, and collaborators become project members.
6. **Posts a banner** on every page ("GitHub is unavailable: this GitLab is now our primary code host…").
7. **Feeds the other GitLab** with push mirrors from every project on the primary. The engine allows the secondary's host through GitLab's outbound-request protection when it is on a private network.
8. **Prints the RPO**: for every repository, the time of the last successful sync from GitHub.

A step that fails for one item (one project, one account, one mirror) is listed under `failures` and alerted, and the rest of the failover continues. The command then exits with code 1. `demote` reverses exactly what the record lists. It exists for drills and for returning to GitHub if it comes back.

4.2 After failover

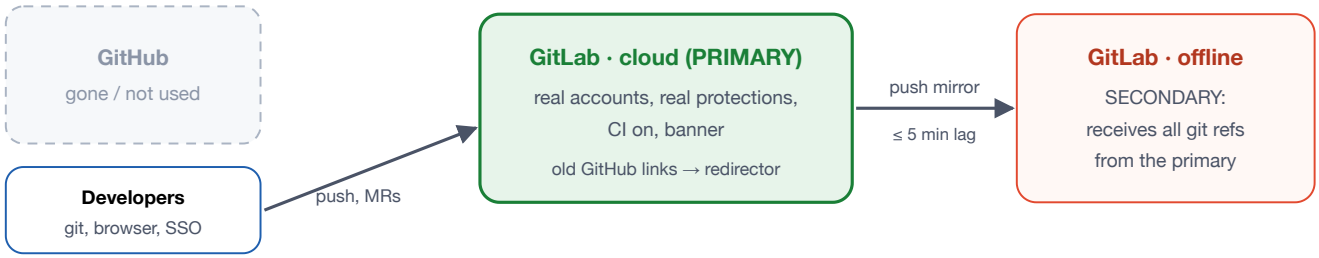


Figure 6. Topology after `promote --primary cloud`. Either target can be promoted. The other becomes the secondary.

> **Two things to know**
>
> 1. GitLab runs a push mirror that covers all branches at most once every 5 minutes, so the secondary trails the primary by up to 5 minutes.
> 2. After failover only **git data** flows to the secondary. Issues and MRs created on the new primary reach the secondary only through GitLab's project export/import. A GitLab-to-GitLab metadata adapter is not built yet (section 9).

4.3 Recovery objectives

Objective	Value
RPO (data loss)	Normally seconds: webhooks trigger syncs immediately. The worst case is the scheduler interval for a missed webhook (metadata 1 h, git 6 h by default). `promote` prints the exact value per repository.
RTO (time to work again)	`promote` takes minutes. The rest is communication: the banner, the remote URL change (a one-line git command or an MDM-pushed `insteadOf` rule), and SSO login.

5 Setup guide

Setup is one command on the engine host. Along the way it gives you one command to run on each GitLab host and, if you choose a GitHub App, two clicks on GitHub. Nothing is configured by hand in GitLab or GitHub, and no IDs, keys or fingerprints are copied around.

#	Step	Where	Who	Time (measured)
1	GitLab on each side (skip if you have one)	each GitLab host	sysadmin	≈ 15 min
2	`./setup.sh`	engine host	anyone with Docker	≈ 5 min
3	the `prepare-host.sh` line that setup prints	each GitLab host, over SSH	someone who may `sudo` there	≈ 4 min each
4	“Create GitHub App” and “Install”	browser	a GitHub org owner	1 min
—	first full copy	runs by itself in the background		≈ 1 API call per object

5.1 What you need

Item	Requirement
GitLab · cloud	Self-managed GitLab CE or EE (Omnibus package or the official Docker image) on a cloud VM: **4 vCPU, 8 GB RAM minimum**, plus about 1 GB for the side-channel agent. Disk: 2–3× the size of all repositories. GitLab.com SaaS is not suitable, because it cannot impersonate users or backdate content.
GitLab · offline	The same, on company hardware. Each instance needs its own host: two GitLabs on one 8 GB machine overload (tested).
Engine host	Any Linux host with Docker and Compose v2: 2 vCPU, 4 GB RAM, disk about 2× the repositories (L1 + L2). Usually placed next to the offline GitLab. Needs outbound HTTPS to GitHub and the cloud GitLab, and SSH to both GitLab hosts. Inbound HTTPS for webhooks is optional (§5.5).
GitHub	An organization owner for two clicks (create and install a read-only GitHub App), or a classic personal access token with `repo, read:org`.
Identity (recommended)	Company SSO (OIDC or SAML) usable by GitLab, so the first login after failover links each person to their account by email.

5.2 Step 1 — GitLab on each side

If a GitLab is already running, skip this step. Otherwise `deploy/gitlab/docker-compose.yml` is a ready template, pinned to the tested version 19.5.3. Set `external_url` and put the root password in `root_password.txt`:

```
cd deploy/gitlab && echo 'a-long-root-password' > root_password.txt
docker compose up -d                # first boot takes 5–10 minutes
```

- Git over SSH is published on port **2222**, so the host's own SSH keeps port 22. The engine's side channel uses the host's SSH.

- Uncomment the three SSO lines (`omniauth_allow_single_sign_on`, `omniauth_auto_link_user`, `omniauth_block_auto_created_users = false`) once your identity provider is configured. Failover then needs no account work.

5.3 Step 2 — Run setup on the engine host

```
git clone <this repository> ghmirror && cd ghmirror/deploy/compose
./setup.sh
```

`setup.sh` needs only Docker. It builds the engine image and asks a few questions. Behind the scenes it creates an SSH key for the side channel, and then it checks every connection and starts the service. A real run, with the long values shortened:

engine-host: ~/ghmirror/deploy/compose

```
$ ./setup.sh
== 1/4 Building the engine image
== 2/4 Configuration
GitHub organization (or user) to mirror: acme
Public HTTPS URL that GitHub can reach for webhooks (Enter = none, poll every 15 min):
https://mirror.acme.com
GitHub access: 'app' creates a read-only GitHub App for you (recommended, 2 clicks);
               'token' uses a classic personal access token (repo, read:org; admin:org_hook
for webhooks).
Which (APP/token): ⏎
  1. Open http://localhost:8765/ in a browser logged in to GitHub as an owner of acme.
  2. Click 'Create GitHub App'. ...
  App ghmirror-acme created (id 1234567).
  3. Install it on acme (all repositories): https://github.com/apps/ghmirror-
acme/installations/new
  Installed (installation id 89012345).
GitLab targets. Prepare each GitLab host once, from here (with an account that may sudo
there):
  ssh <admin>@<gitlab-host> sudo bash -s -- --pubkey "'ssh-ed25519 AAAAC3Nz… ghmirror-
engine'" < ~/ghmirror/deploy/compose/prepare-host.sh
Paste the ghmirror-target line for 'cloud' (or 'skip'): ghmirror-
target:eyJ1cmwiOiAiaHR0cHM6Ly9…
  cloud: https://gitlab-cloud.acme.com  token ok, admin
Paste the ghmirror-target line for 'offline' (or 'skip'): ghmirror-
target:eyJ1cmwiOiAiaHR0cHM6Ly9…
  offline: https://gitlab.lan.acme.com  token ok, admin
Slack-compatible alert webhook URL (Enter = none): https://hooks.slack.com/services/…
Wrote ghmirror.yaml, .env, identities.yaml, secrets/.
== 3/4 Checking GitHub, GitLab and the side channels
github: acme (org), repos visible: 214
target cloud: GitLab 19.5.3, group acme (id 2), bot github-mirror, mode mirror, side channel
ok (Rails 19.5.3)
target offline: GitLab 19.5.3, group acme (id 2), bot github-mirror, mode mirror, side
channel ok (Rails 19.5.3)
== 4/4 Starting the service and queueing the first full copy
ghmirror is running. The first copy proceeds in the background.
```

The prompts, messages and order are those of the tested build. The organization name, URLs and counts are illustrative. The live test mirrored a personal account (§8.3).

5.4 Step 3 — Prepare each GitLab host

When setup reaches “GitLab targets”, open a second terminal on the engine host and run the printed line once per GitLab host, replacing `<admin>@<gitlab-host>`. The script travels over your own SSH session, so nothing needs to be installed first. Then paste the last line it prints into setup.

engine-host: second terminal

```
$ ssh admin@gitlab.lan.acme.com sudo bash -s -- --pubkey "'ssh-ed25519 AAAAC3Nz… ghmirror-engine'" < prepare-host.sh
[ghmirror] GitLab: omnibus
[ghmirror] created Unix user ghmirror
[ghmirror] authorized the engine's SSH key
[ghmirror] sudo rule: /opt/gitlab/bin/gitlab-rails, /usr/bin/gitlab-rails, /bin/gitlab-rails, /usr/bin/sh, /bin/sh, /usr/bin/pkill, /bin/pkill
[ghmirror] creating the bot account and token (Rails boot, please wait)...
[ghmirror] bot github-mirror ready on GitLab 19.5.3 at https://gitlab.lan.acme.com (token expires in 365 days)

=== GitLab host ready. Paste this line into 'ghmirror init' (it contains the bot's token):
===
ghmirror-target:eyJ1cmwiOiAiaHR0cHM6Ly9naXRsYWIubGFuLmFjbWUuY29tIiwgInRva2VuIjog…
```

The script is idempotent, so running it again is harmless and issues a fresh token. It does four things:

1. **Side-channel login.** It creates a Unix user `ghmirror` that can log in only with the engine's key, without port forwarding or a terminal.
2. **Exact sudo rule.** `/etc/sudoers.d/ghmirror` allows `gitlab-rails`, `sh` and `pkill` without a password and nothing else. Every path sudo might resolve is listed. For a Docker-hosted GitLab the rule is `docker exec -i <container> *` instead, and the container is found automatically.
3. **Mirror bot.** It creates the GitLab administrator `github-mirror` and a token with the scopes `api, sudo, admin_mode, read_api, write_repository`.
4. **One line for setup.** It prints the URL, token, SSH login and the host's SSH key. The engine therefore trusts exactly this host, and nobody has to confirm a fingerprint. The line stays under 1024 characters, the most a terminal accepts in one paste.

Option	Use when
`--container NAME`	More than one GitLab container runs on the host.
`--ssh-host HOST`	The engine reaches the host under another name or IP than `hostname -f`.
`--ssh-port PORT`	The host's own sshd is not on port 22.
`--bot NAME`, `--token-days N`	Another bot username, or a token lifetime other than 365 days. Rotate by re-running the script and setup before it expires.

> **Security note**
>
> `gitlab-rails runner` can run any code with GitLab's privileges, and `sudo sh` is a root shell. The side channel is therefore as powerful as root on the GitLab host. Treat the engine host and `secrets/id_ed25519` like GitLab's root account: a dedicated host, restricted logins, and the key used for nothing else. If that is not acceptable, remove `side_channel` from `ghmirror.yaml`. The engine still works, and only the fields in §3.5 lose their original values.

5.5 Step 4 — GitHub access and webhooks

GitHub App (recommended). Setup builds the App definition for you: read-only access to contents, metadata, issues, pull requests, discussions and administration, members on the organization, and the events the engine handles. It then sends your browser to GitHub with everything filled in. Click *Create GitHub App*, then *Install*. Setup receives the App's ID, private key and webhook secret directly from GitHub, and detects the installation by itself.

- **Engine on a remote server?** Run `ssh -L 8765:localhost:8765 <engine-host>` from your laptop first, so the browser can reach setup. If the page GitHub returns to does not load, copy its address from the browser into setup instead.
- **Token instead:** answer `token` and paste a classic PAT (`repo, read:org` ; add `admin:org_hook` and setup also creates the organization webhook). For a personal account, a token is the simplest choice.

Webhooks make changes appear within seconds. GitHub must reach `https://<your-host>/webhook`, so route that path to `127.0.0.1:8080` on the engine host with your reverse proxy or a tunnel. With no public URL, answer Enter: the engine then polls every 15 minutes and nothing else changes.

5.6 What setup leaves behind

File (in `deploy/compose/`)	Content
`ghmirror.yaml`	The configuration, with no secrets in it (they are `${VAR}` references). Edit it freely, for example `source.repos.include/exclude` or `guard` . Appendix B lists every key.
`.env` (mode 600)	Tokens, the App ID, the webhook secret, a random database password, the alert URL.
`identities.yaml`	GitHub login → company email. Setup lists every organization member as a commented line, so you only fill in addresses. Needed before a failover, not before mirroring.
`secrets/`	`id_ed25519` (side-channel key), `known_hosts` (the GitLab hosts' keys), `github-app.pem` . The container reads them as the unprivileged user, whoever owns them on the host.
`prepare-host.sh`	The host script from step 3, kept for adding or re-preparing a GitLab later.

Re-running is safe. `./setup.sh` keeps existing secrets and targets, and asks only what is missing. Run it again to add a GitLab, rotate a token or switch from token to App. **Unattended** installs answer every question with flags:

```
GITHUB_TOKEN=ghp_… ./setup.sh --yes --owner acme --github-auth token \
    --webhook-url https://mirror.acme.com --alert-webhook https://hooks.slack.com/services/… \
    --target "cloud=ghmirror-target:eyJ…" --target "offline=ghmirror-target:eyJ…"
```

5.7 The first copy

Setup queues a full pass for every repository, and the service works through it: git first, then issues, pull requests and reviews, then members. It is **idempotent and resumable**. A restart or a crash continues where it stopped, and nothing is duplicated. Until a repository's first copy is done, the parity report lists it as `pending` rather than as drift, so a new install does not raise false alarms.

```
docker compose run --rm engine status      # queue: pending / running / done
docker compose logs -f engine              # live progress
docker compose run --rm engine verify      # "drift_count": 0 when the copy is complete
```

Expect about one GitHub API call per object. A GitHub App installation gets at least 5,000 calls per hour, more for large organizations and on Enterprise Cloud. For very large organizations, narrow `source.repos.include` first and widen it in steps.

5.8 Steady state

The `engine` service runs `ghmirror run`: webhook receiver, workers and scheduler in one process. No cron is needed. Default schedule (without webhooks, setup sets `git` and `meta` to 900) (seconds, configurable under `schedule:`):

Pass	repos	meta	git	org	full	verify	migration
Interval	3600	3600	21600	21600	86400	86400	604800

5.9 Monitoring and alerts

Endpoint / metric	Alert when
`GET /healthz`	Not 200 (liveness).
`ghmirror_meta_age_seconds`	Greater than 2 × `schedule.meta` (sync is lagging).
`ghmirror_jobs{status="dead"}`	Greater than 0 (a job failed 5 times).
`ghmirror_drift_count`	Greater than 0 after the nightly verify.
`ghmirror_halted`	Equals 1: the deletion guard stopped all writes. **Page a human.**
`ghmirror_alerts_total`	Trend. Every alert is also posted to `alerts.webhook_url`.

`ghmirror status` (or `GET /status`) prints the mode, queue, last parity report and recent alerts.

6 A tour of the mirror

Every screenshot in this section is from the end-to-end test. A private GitHub repository with pull requests, reviews, issues, labels, milestones, releases, bots and a deleted repository was mirrored into two GitLab CE 19.5 instances. Unless a caption says otherwise, they show the **cloud** GitLab as an administrator sees it. The offline GitLab is identical (§6.8).

6.1 Pull requests keep their numbers and dates

GitHub PR #3 is GitLab merge request !3, and issue #1 is issue #1. Old links, commit messages such as `(#7)`, and references in chat and tickets therefore still point at the right item. Authors, creation and merge times, and merge state are copied exactly:

GitHub	Title	State on GitHub	Created (UTC)	Merged (UTC)	GitLab	State on GitLab	Match
PR #3	Merge-commit PR	merged	11:00:52	11:01:09	!3	merged	exact
PR #4	Squash PR	merged (squash)	11:00:54	11:01:17	!4	merged	exact
PR #5	Rebase PR	merged (rebase)	11:00:55	11:01:25	!5	merged	exact
PR #6	Closed PR	closed	11:00:57	—	!6	closed	exact
PR #7	Open PR	merged later	11:00:58	11:58:56	!7	merged	exact
PR #9	Late PR	open	11:58:48	—	!9	opened	exact
Issue #1	Crash on startup (config loader)	closed, 2 comments	11:00:45	—	#1	closed, 2 notes	exact
Issue #2	Docs typo	closed, 1 comment	11:00:50	—	#2	closed, 1 note	exact
Issue #8	New issue after first sync	open	11:58:46	—	#8	opened	exact

All on 31 October 2026, all by `shoubhiksaha` on both sides. There is no PR #8 and no issue #3–#7 or #9: GitHub numbers issues and PRs from one shared sequence, and GitLab leaves exactly the same gaps.

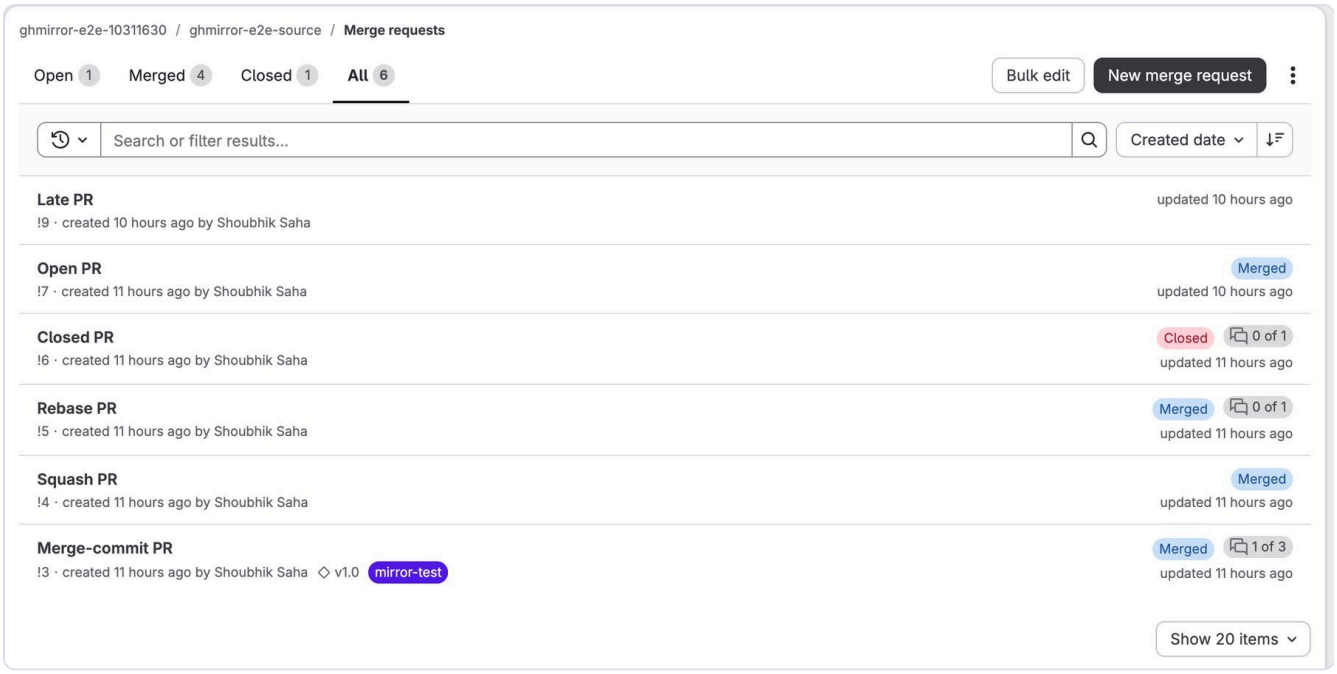


Figure 7. Merge requests of the mirrored project: !3 to !9 with GitHub's titles, authors and states. !3 carries its label `mirror-test` and milestone `v1.0`.

6.2 Reviews, threads and reactions

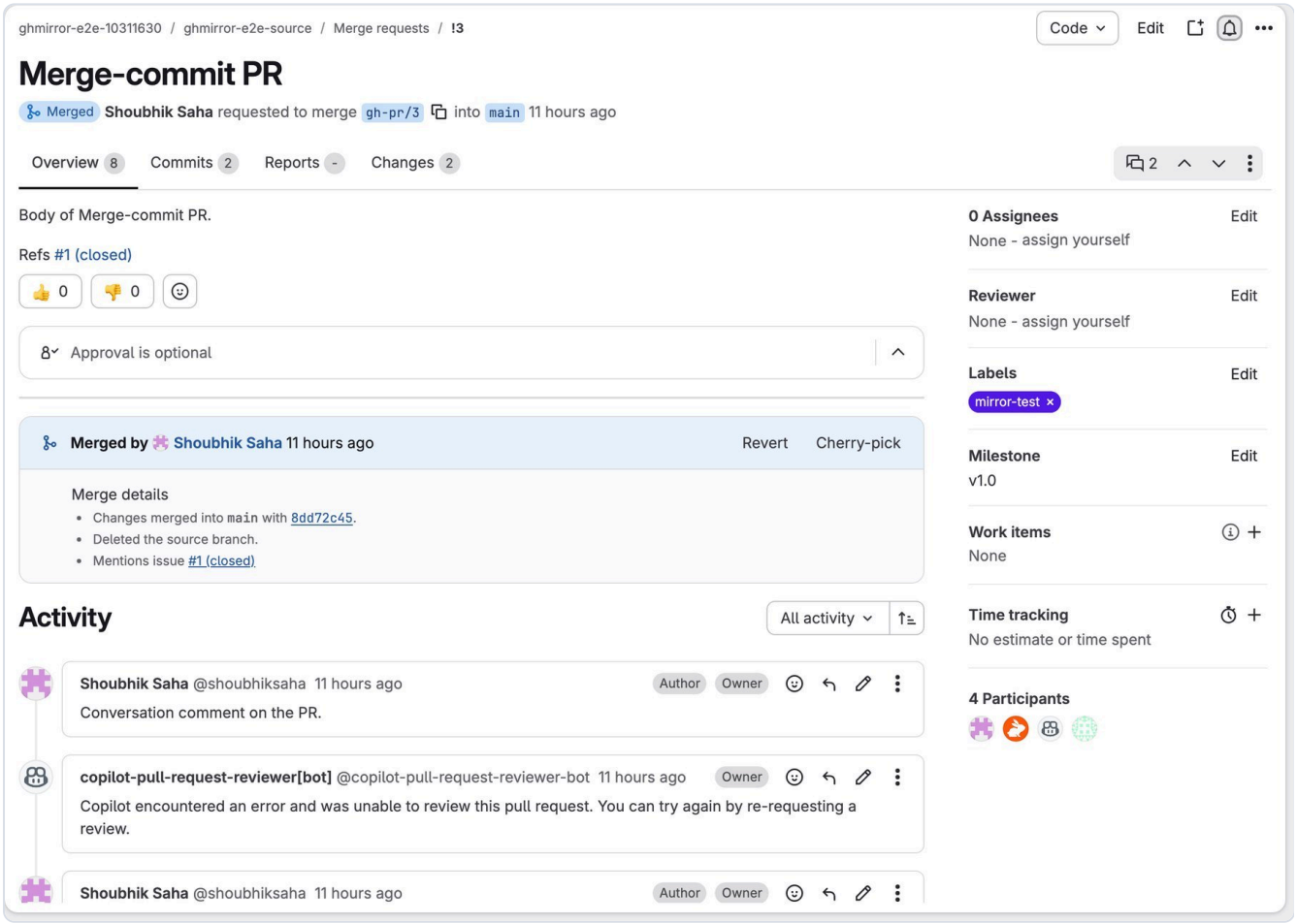


Figure 8. MR !3. The author is the person's mirror account, not a bot, and the timestamps are GitHub's. “Merged by Shoubhik Saha” is set through the side channel. The description, the review summary and comments from bots such as Copilot appear in order.

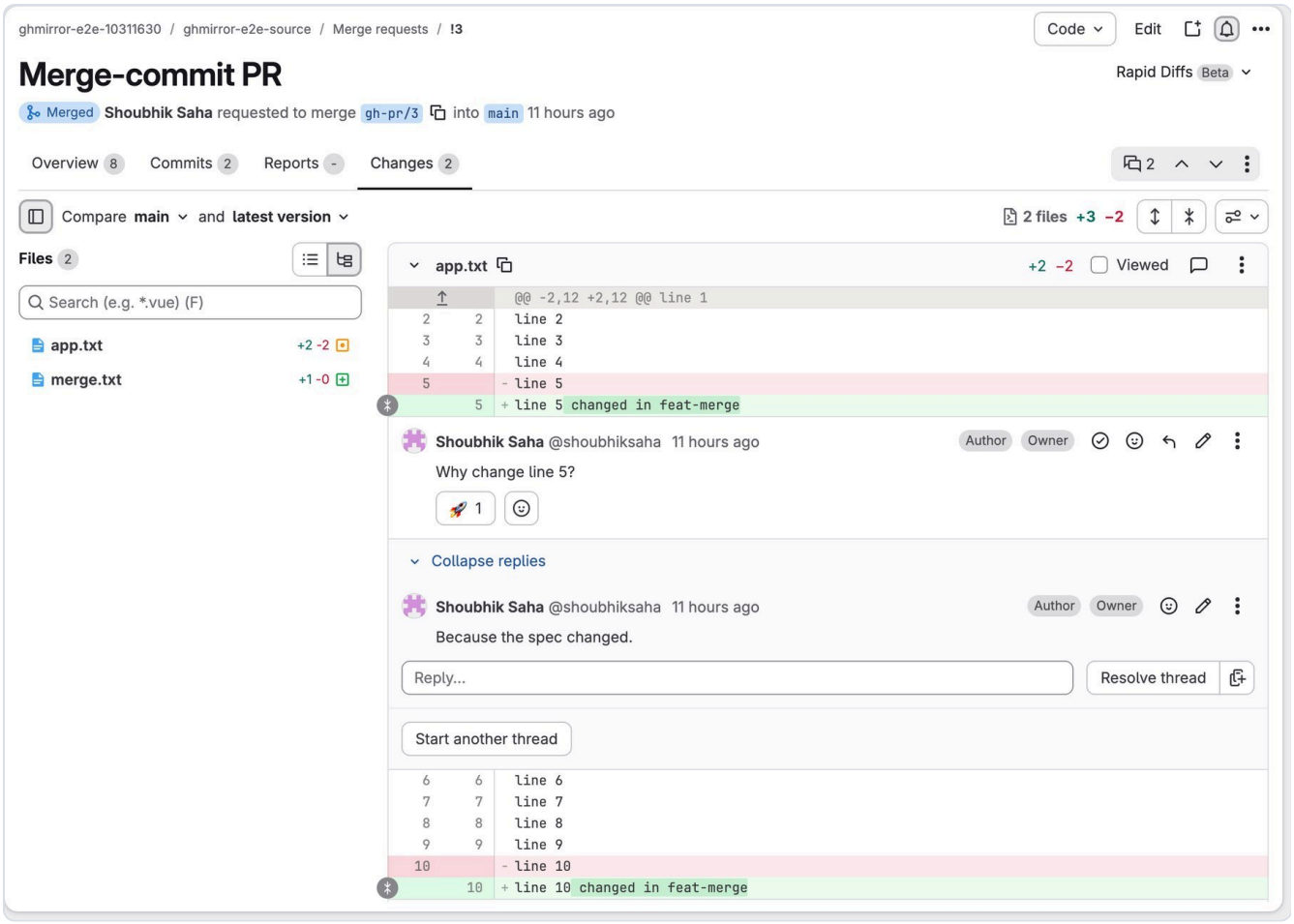


Figure 9. The same review comment sits on the same line of the same diff: “Why change line 5?” on `app.txt` line 5, with its reply and the 🚀 reaction. The diff is GitHub's exact base…head, rebuilt from hidden refs even though the source branch was deleted on GitHub after merging.

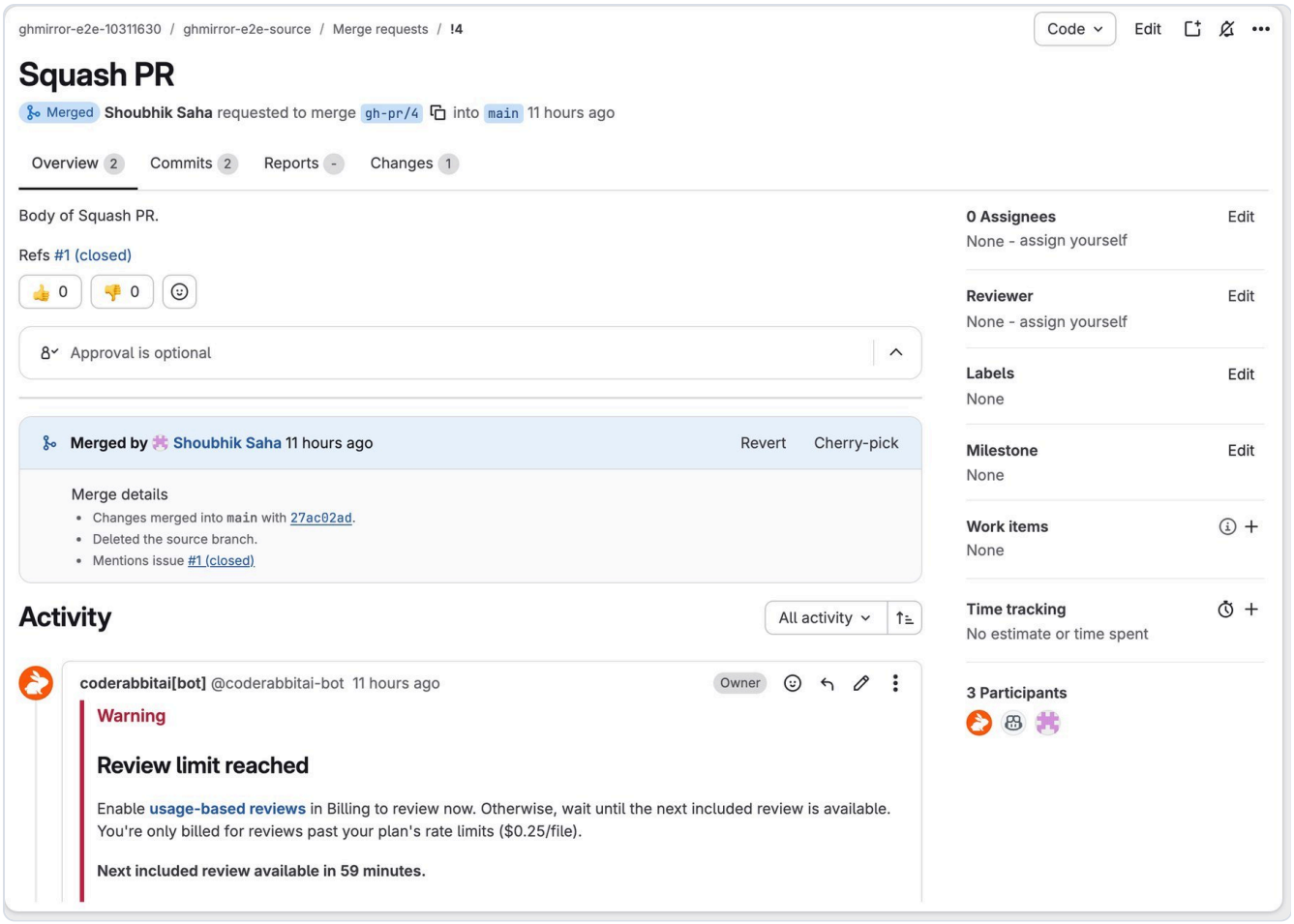


Figure 10. MR !4 was a **squash** merge on GitHub. GitLab's API cannot mark such an MR as merged, so the side channel records it as merged, with the right merger and time.

6.3 Issues

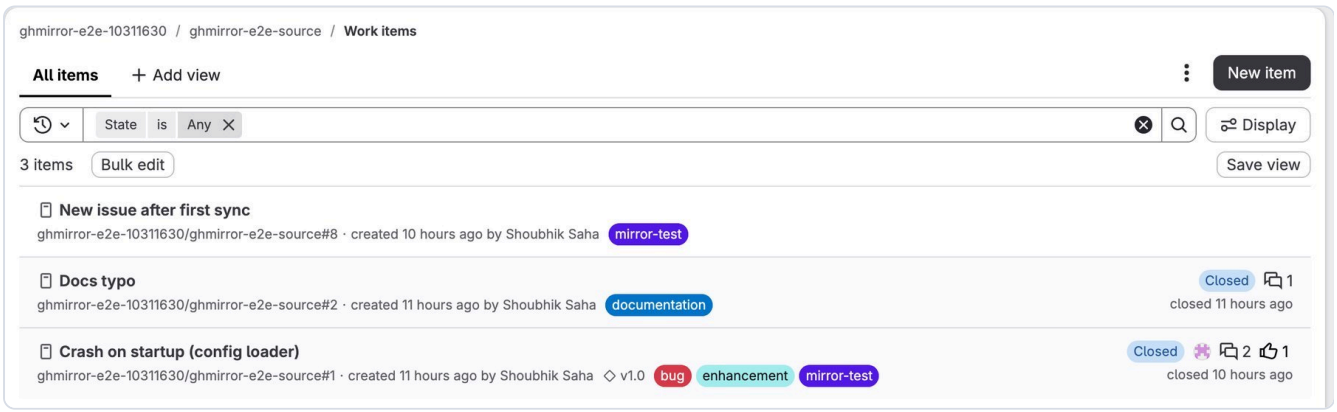


Figure 11. Issues with their GitHub numbers, states, labels (colours included) and milestone.

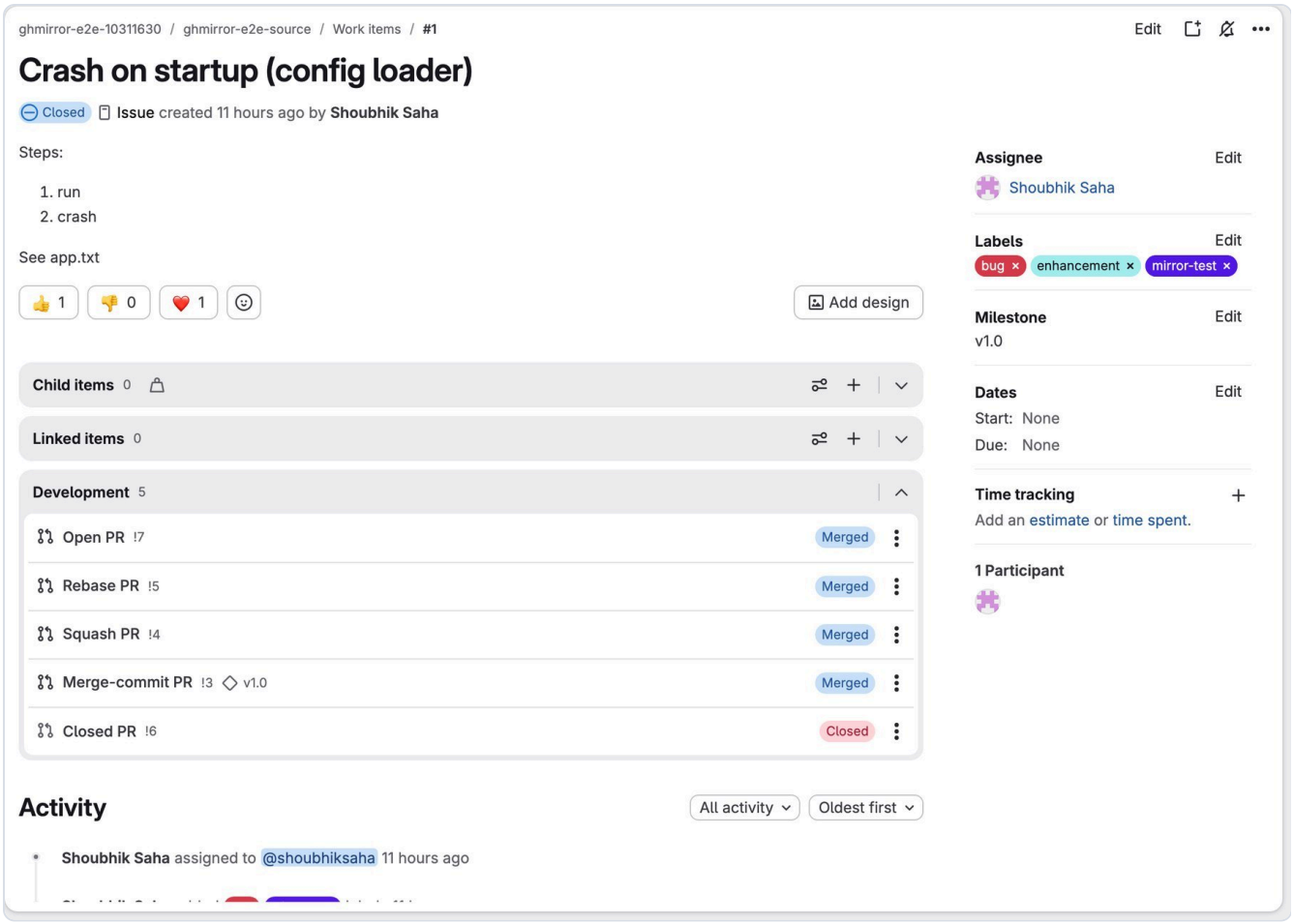


Figure 12. Issue #1 with reactions, assignee, labels and milestone. The *Development* panel lists the MRs that reference it, which is GitHub's "linked pull requests".

6.4 Repository: branches, tags, releases, milestones

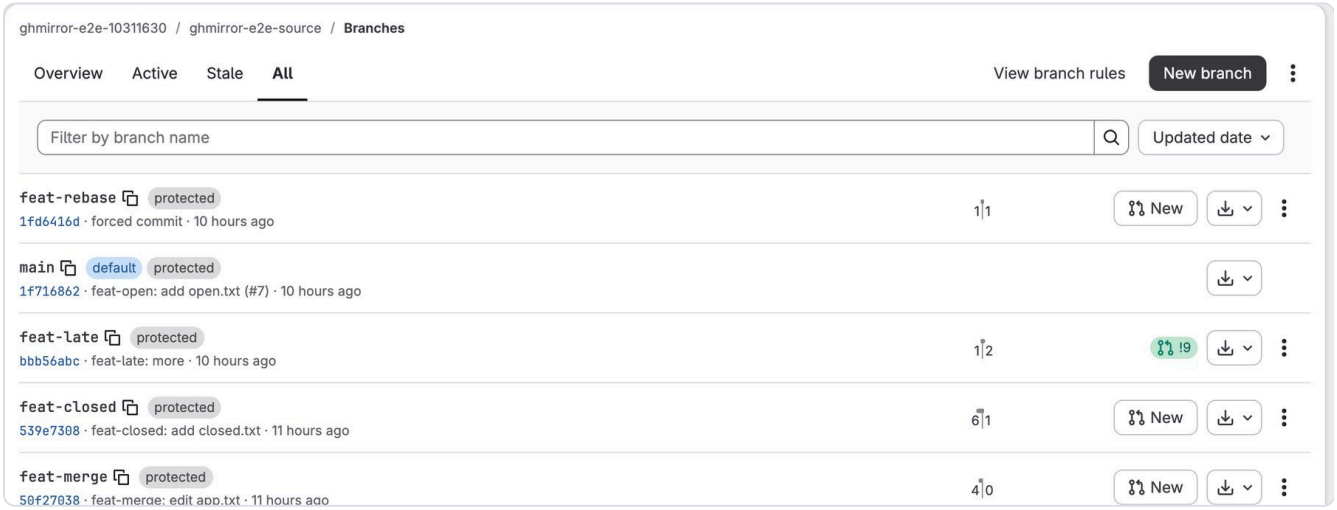


Figure 13. All branches, at GitHub's exact commits.

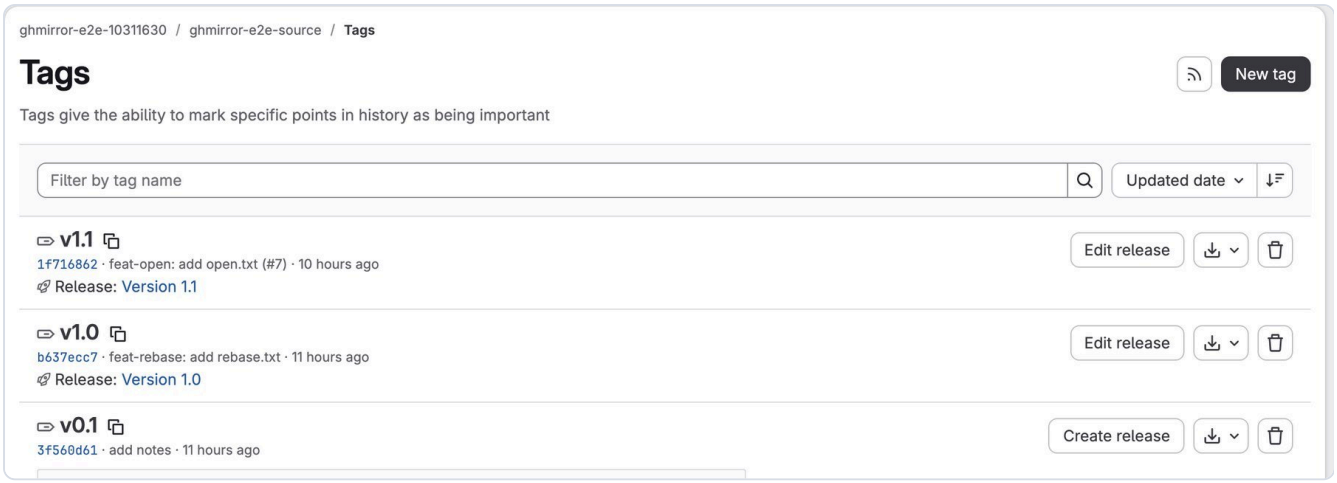


Figure 14. Tags, including annotated ones, linked to releases.

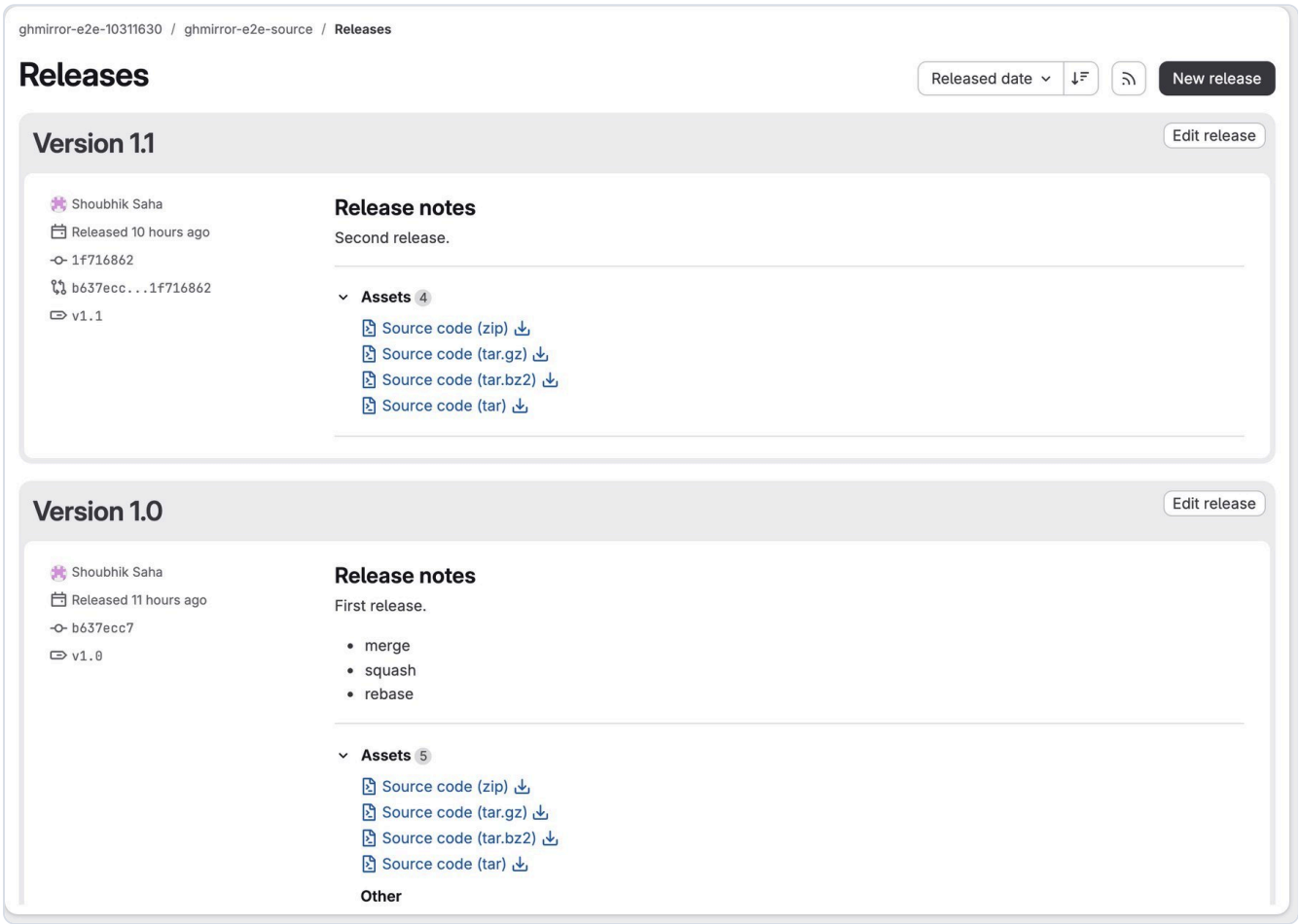


Figure 15. Releases with their notes, author and dates. Uploaded release assets are copied into GitLab's package registry and attached to the release as links.

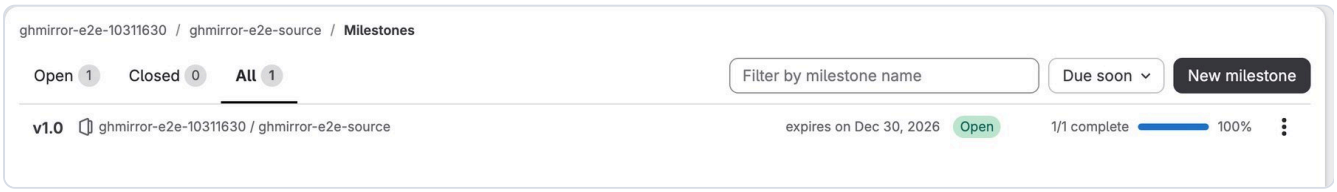


Figure 16. Milestone `v1.0` with its due date and progress.

6.5 Nobody can change the mirror by accident

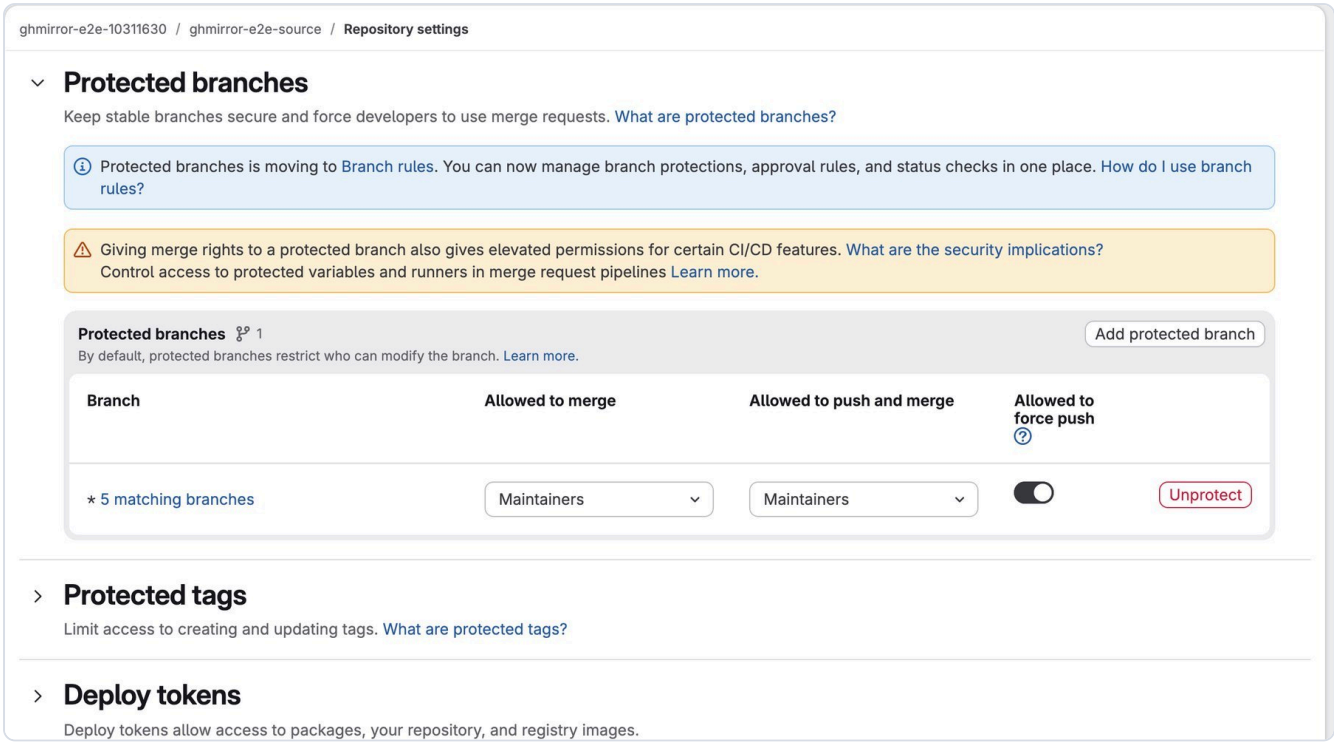


Figure 17. In MIRROR mode, every branch matches a protected rule `*`. People cannot push or merge, and only the mirror bot writes. Promote replaces this rule with the GitHub branch protections (§4).

6.6 Mirror accounts

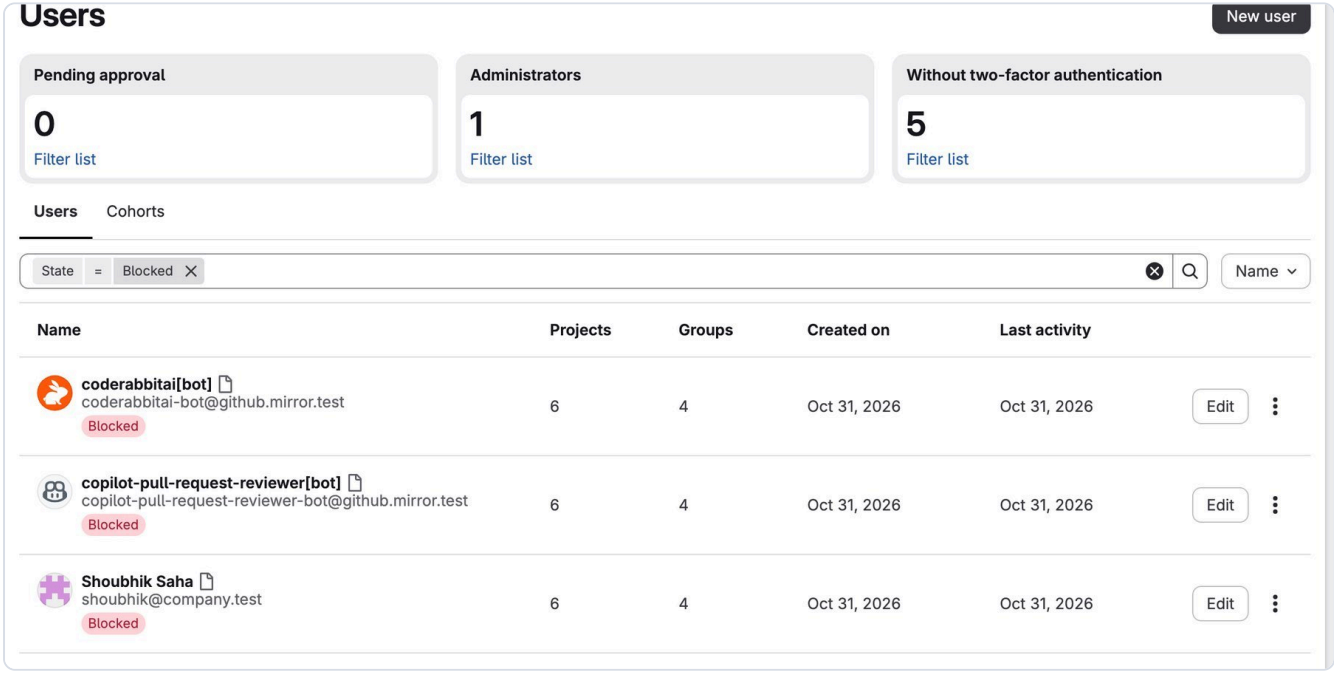


Figure 18. Admin › Users, filtered on *Blocked*. Every GitHub person and bot has an account, blocked while GitHub is primary. Shoubhik Saha already has the company email from `identities.yaml`. The bots keep placeholder addresses.

6.7 Deleted on GitHub, kept on GitLab

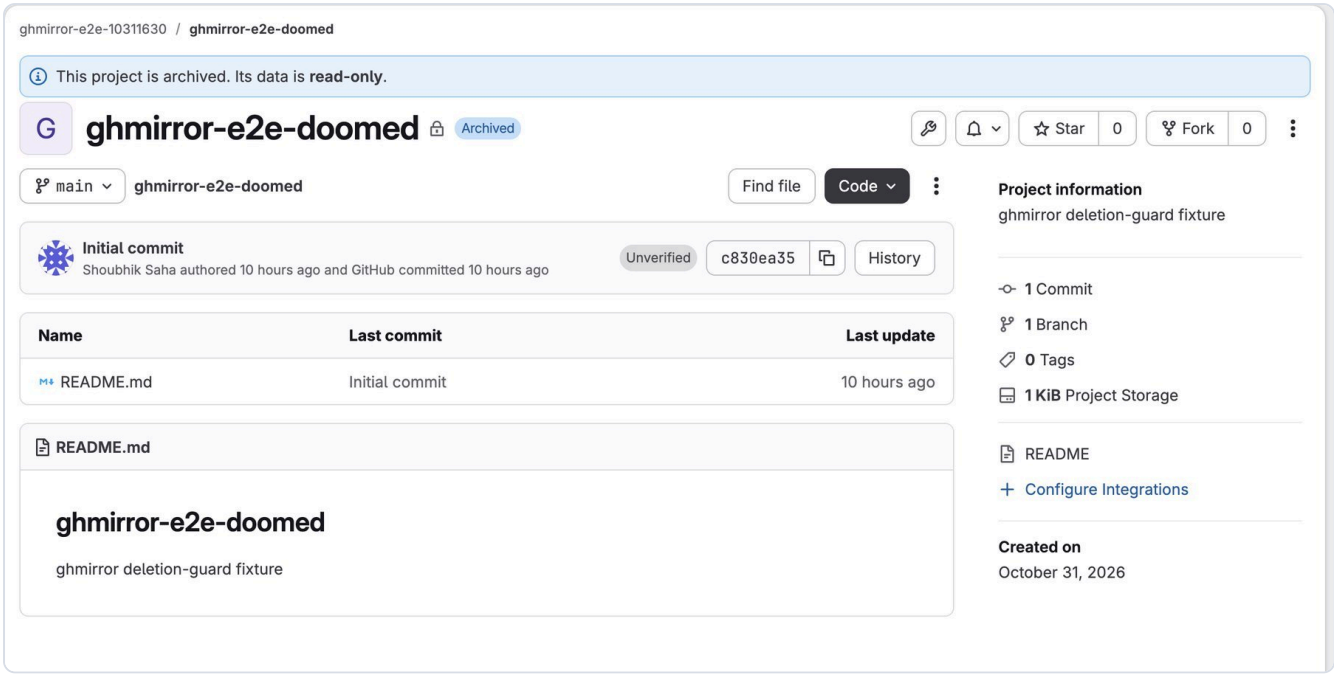


Figure 19. The repository `ghmirror-e2e-doomed` was deleted on GitHub during the test. The mirror does not follow deletions: the GitLab copy is **archived** (read-only), keeps all its data, and an alert is raised. This holds on both GitLabs.

6.8 The offline copy

The offline GitLab receives the same objects from the same engine, independently. It is not a copy of the cloud GitLab, so if one GitLab is damaged the other is not affected.

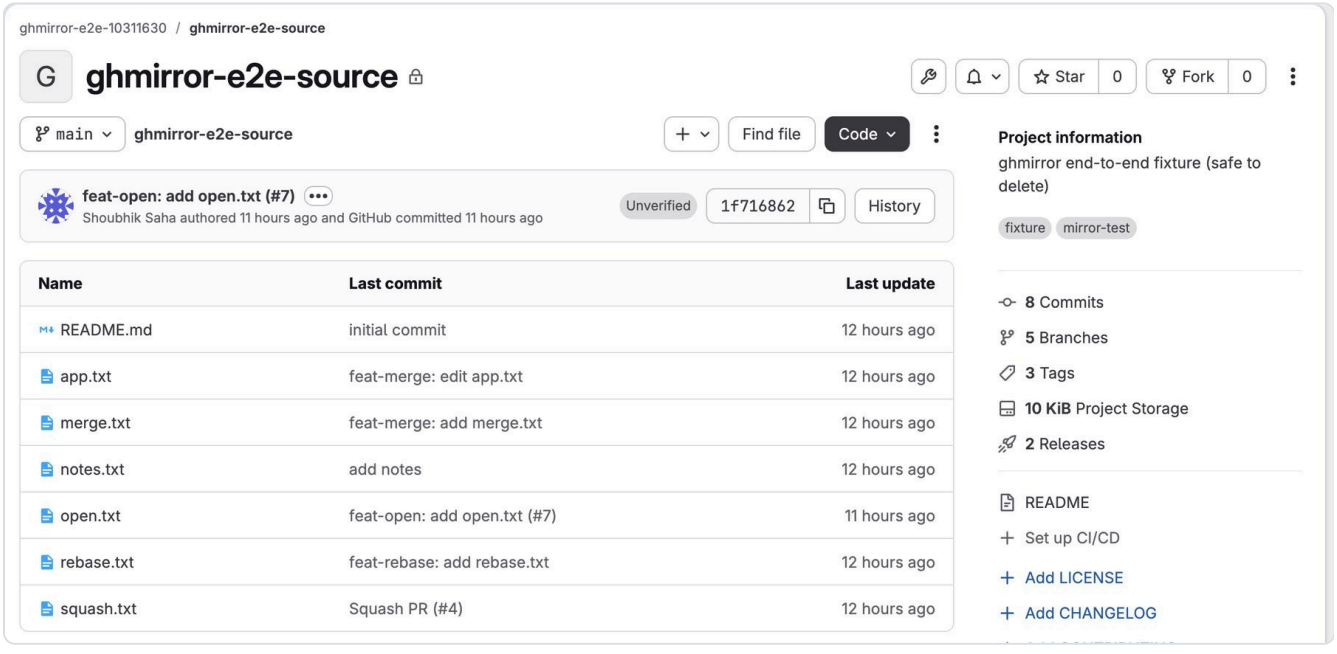


Figure 20. Offline GitLab: the same project, 8 commits, 5 branches, 3 tags, 2 releases, head `1f716862`.

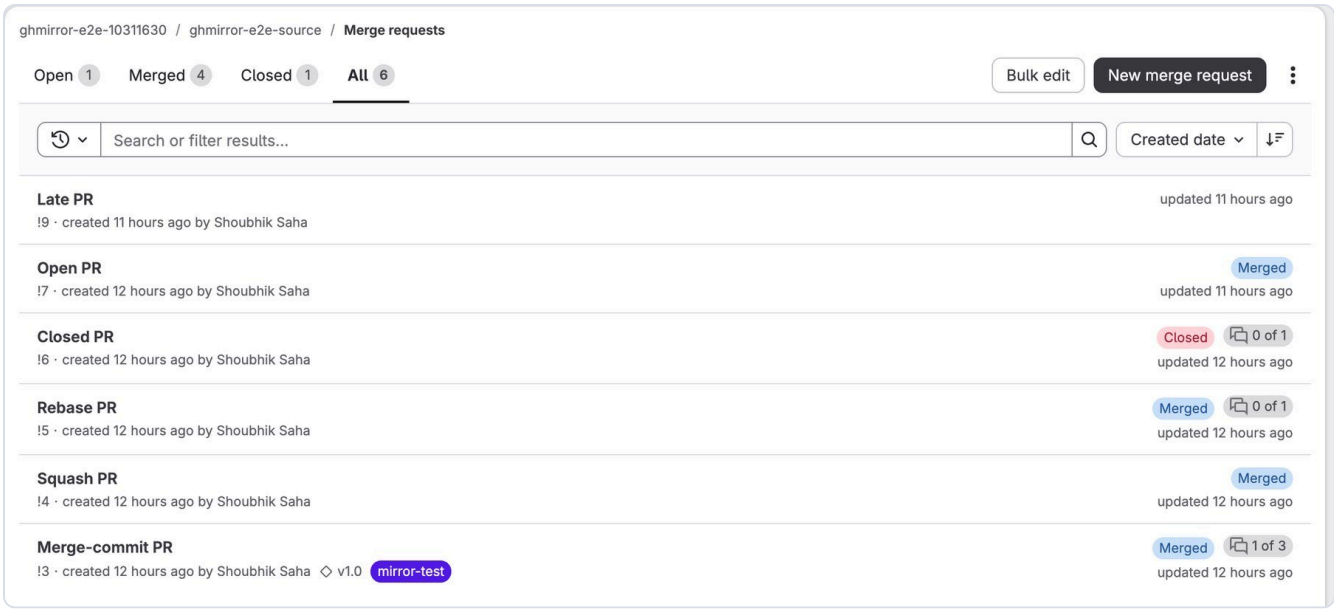


Figure 21. Offline GitLab: the same merge requests !3 to !9.

6.9 After a failover (promote)

These screenshots come from the failover drill. `ghmirror promote --primary cloud` was run, then `demote` afterwards.

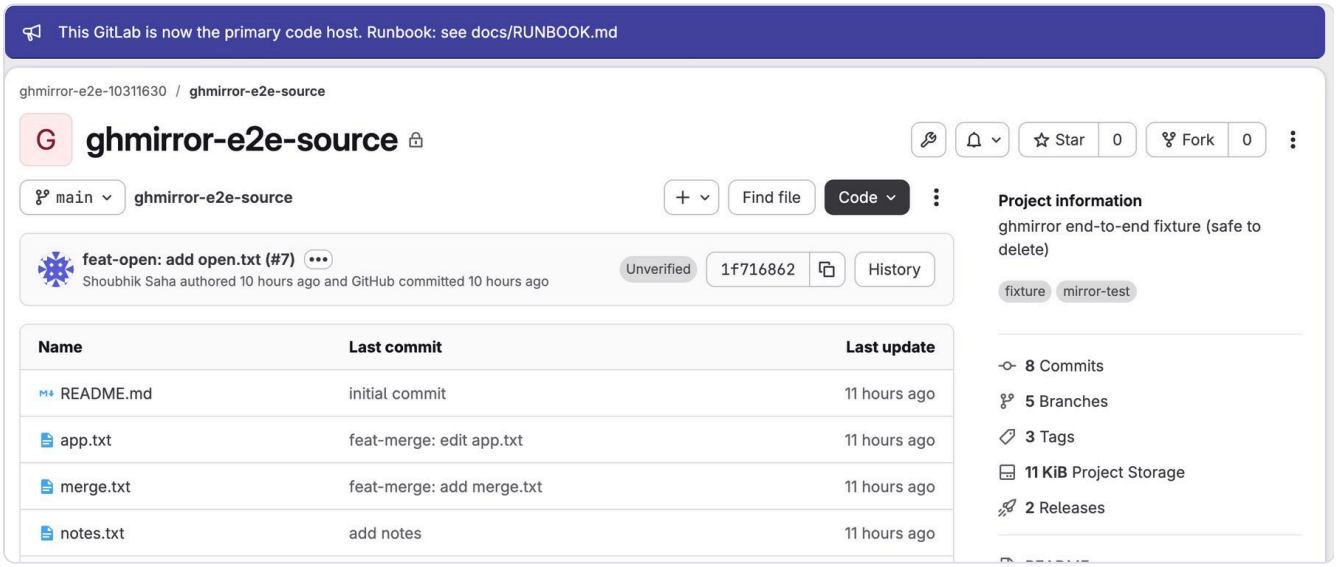


Figure 22. Everyone sees a broadcast banner. CI/CD (*Build* in the menu) is now enabled, and GitHub's branch protections replace the mirror-only rule.

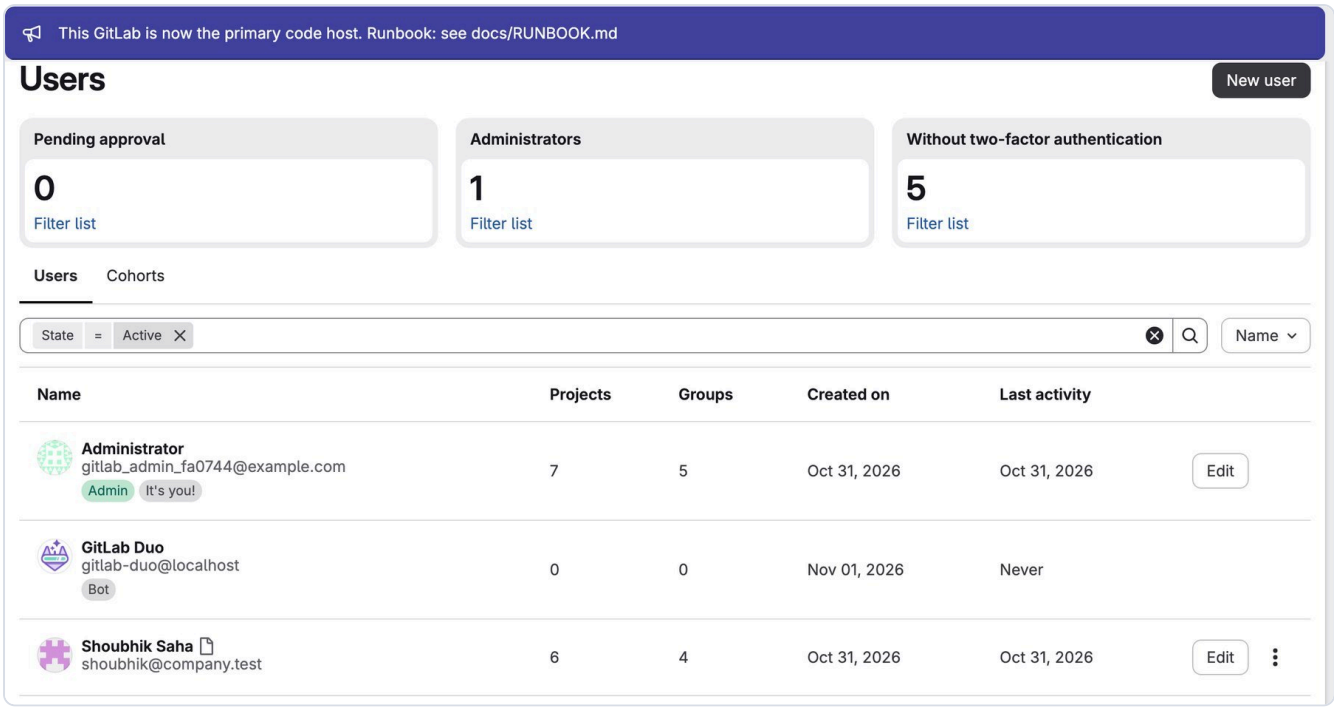


Figure 23. Shoubhik Saha's account is now **active** with the company email, so the first SSO login links to it and the person sees their own history. The bot accounts stay blocked.

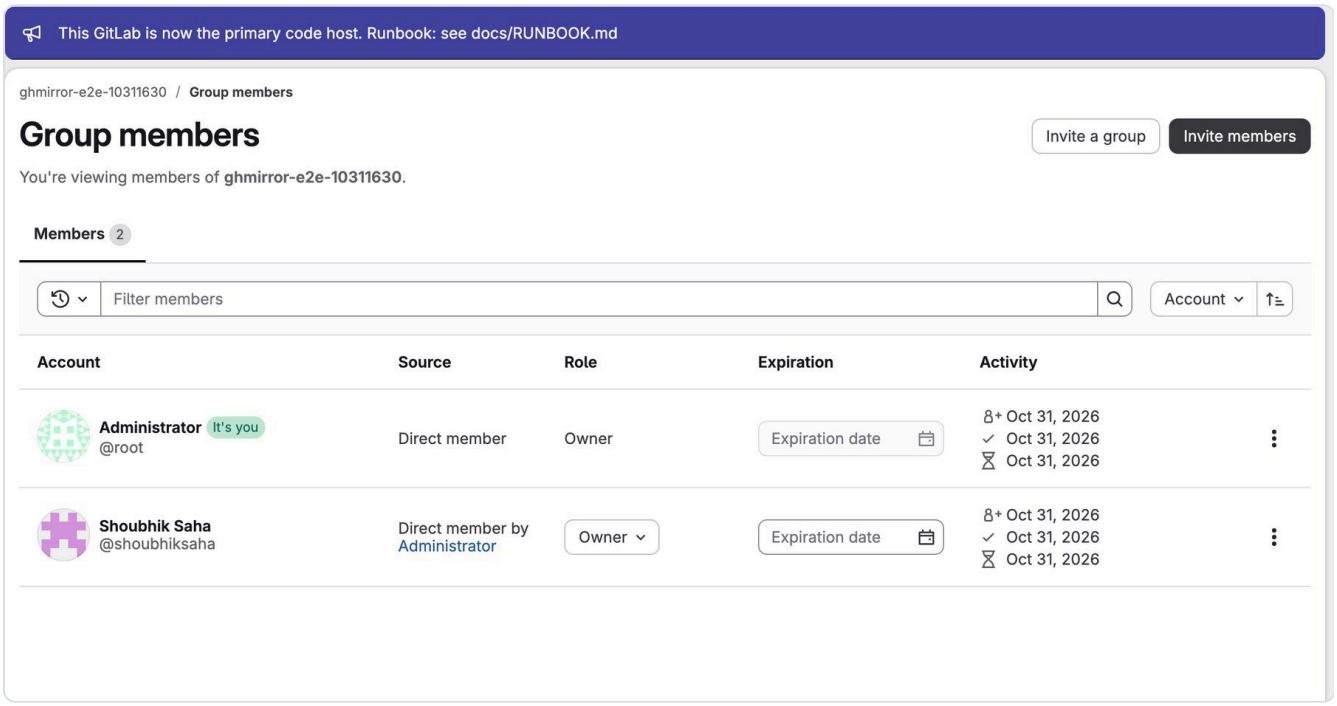


Figure 24. Group members after promote: the person has the role GitHub gives them. The test used a personal account, whose owner becomes Owner. In an organization, owners become Owners and members get the organization's base permission.

7 Operations runbook

In steady state the mirror needs no attention. This chapter covers the few situations that need a person. Run every command from `deploy/compose/` on the engine host, as `docker compose run --rm engine <command>`.

7.1 Routine checks

When	Check	Expected
Continuously	Alerts in the alert channel; the `/metrics` signals in §5.9.	No `error` alerts, `ghmirror_halted 0`.
Daily (automatic)	The nightly `verify` parity report.	`ghmirror_drift_count 0`.
Weekly	`status`: queue, dead jobs, last parity report, recent alerts.	No `dead` jobs.
Monthly	Failover drill (§7.4).	Drill passes, then 0 drift after demote.
Before expiry	GitLab bot tokens expire after 365 days. Re-run `prepare-host.sh` and paste the new line into `./setup.sh`.	`check` ok.

7.2 When the deletion guard halts

The engine stops **all** writes when the GitHub owner returns 404, or when more repositories or issues vanish in one pass than `guard.max_disappear_*` allows. A single deleted repository does not halt anything: it is archived on GitLab (Figure 19). On `ghmirror_halted 1`:

1. Look at GitHub: the organization page, githubstatus.com, the App installation.
2. **GitHub is fine**, for example you moved many repositories on purpose: run `guard reset`. Mirroring resumes and catches up.
3. **GitHub is gone or compromised:** do **not** reset. The GitLab copies are frozen in their last good state. Go to §7.3.

7.3 Failover checklist (GitHub lost)

1. **Decide the primary.** Usually `cloud`, which is reachable from everywhere. Use `offline` if the cloud is affected too.
2. **Promote:** `promote --primary cloud`. Type `promote` to confirm. It prints the state, the users activated, the mirrors and the RPO (last successful sync per repository). Exit code 1 means some steps failed, and each failure is listed. Fix the cause, then run `demote` and `promote` again.
3. **Redirect old links:** `docker compose --profile failover up -d redirector`, and point bookmarks and documentation links (or a vanity DNS name) at it. Old `github.com/<org>/<repo>/pull/7` links then land on `!7`.

4. **Tell developers** to run one command per clone, or push `url.<new>.insteadOf <old>` through device management:

```
git remote set-url origin git@gitlab.acme.com:acme/<repo>.git     # same SSH keys work
```

5. **Sign-in:** with SSO, the first login links each person to their account automatically (§4). Without SSO, an administrator sends password resets from Admin › Users.

6. **CI:** CI/CD is enabled on promote. Re-create secrets as GitLab CI variables; GitHub never exposes them.

> **The secondary trails by up to 5 minutes**
>
> After promote, the other GitLab is updated by push mirroring, which GitLab runs at most once every 5 minutes per project. Only git data flows to it. Issues and MRs created after failover exist on the primary only.

7.4 Monthly drill

Practise on a snapshot of the offline instance or on a scratch GitLab, never on the live cloud mirror: `promote` → two engineers clone, branch, open an MR, approve, merge and tag → `demote` → delete the merge requests and issues made during the drill → `sync --full` → `verify` must report 0 drift. Steps 11–13 of `e2e/run_e2e.sh` automate exactly this (§8.1); chapter 14 shows a recorded drill.

The cleanup matters. Right after `demote`, `verify` reports the drill's work: branches that moved and merge requests that exist only on GitLab. ghmirror never deletes them itself. The next sync resets the branches to GitHub, and any commit that exists only on GitLab is first saved in the engine's git mirror under `refs/ghmirror-backup/<time>/target/`. Merge requests and issues stay until a person deletes them. A drill MR left in place (say !8) also takes the number that GitHub's next PR #8 needs. This is why drills belong on a snapshot.

7.5 Rebuilding a lost GitLab

Situation	Procedure
GitHub still available	New empty GitLab → `prepare-host.sh` → `./setup.sh` (paste its line). The first copy rebuilds it exactly, like the original seed.
GitHub gone, the other GitLab alive	Move the group from the surviving GitLab with GitLab's own group and project export/import, or Direct Transfer. Issues, MRs, notes and history are kept.
Both GitLabs gone	Code: `git push --mirror` from the bare mirrors in the `mirrors` volume. Metadata: preserved as JSON in the `archive` volume, plus GitHub's own migration archives if a `migration_token` is set. Replaying JSON into GitLab is not automated (§9).

Back up the engine host's `mirrors` and `archive` volumes with your normal backup tool. With them, the code and the full record survive even if every hosted copy is lost.

7.6 Everyday changes

Task	How
Mirror more or fewer repositories	Edit `source.repos.include / exclude` in `ghmirror.yaml`, then `docker compose up -d engine`. Repositories that leave the selection are kept on GitLab.
Add a GitLab target	`prepare-host.sh` on the new host, then `./setup.sh --target-names cloud offline dr` and paste the new line. Existing targets are kept.
Add or correct a person's email	Edit `identities.yaml` and restart the engine. New accounts get the address when they are created, and promote applies the file to every account.
Upgrade GitLab	Upgrade the offline instance first, then run `check` and `verify`. The side channel uses GitLab internals, so §8 lists the version tested (19.5.3).
Upgrade the engine	`git pull && ./setup.sh`. It rebuilds the image, offers your previous answers as defaults (`--yes` keeps them all) and restarts the service on the new image.

8 Verification and test results

Three layers of tests: an end-to-end run against a real GitHub account and two real GitLab instances, a from-scratch test of the setup path, and unit tests.

8.1 End-to-end test

`e2e/run_e2e.sh` builds a fixture on GitHub with every object type: merge, squash and rebase merges, a closed PR, an open PR, reviews with line threads, replies, reactions, bot comments, labels, milestones, tags, releases and a repository to delete. It mirrors the fixture into two GitLab CE 19.5.3 instances, changes things on GitHub, and asserts the results object by object. Final run:

#	Step	Result
1	GitHub fixture created; `check` (both targets, side channels)	pass
2	Seed sync of both targets, then 93 object assertions and a parity report with 0 drift	pass
3	Idempotent re-run: nothing changed, audit count unchanged (12 → 12)	pass
4	Service up; a webhook with a bad signature rejected (401); a duplicate delivery ignored	pass
5	Changes on GitHub picked up by webhooks; 115 object assertions; parity 0 drift	pass
6	Repository deleted on GitHub → archived and kept on both GitLabs; the service does not halt for one deletion	pass
7	Owner vanishes → halt (exit 3, writes frozen at 54, later syncs refused); `guard reset`	pass
8	`promote`: person active with company email, bots blocked, CI on, push mirror to offline	pass
9	Developer pushes, opens MR !12 and merges it on the promoted GitLab	pass
10	Secondary receives the primary's new commits through the push mirror	pass *
11	`demote`: account blocked again, bot-only protection restored	pass
12	Re-sync after the drill restores exact parity on both targets	pass
13	Redirector: `github.com/…/pull/3` → `…/merge_requests/3`	pass
14	Weekly GitHub migration archive	skip

* The first attempt at step 10 checked after 8 minutes and failed. GitLab pushes a mirror at most once every 5 minutes and backs off after an early attempt (ROADBLOCKS E15), so the test now waits up to 8 minutes, and the repeat with MR !12 passed. The failover steps (8–13) were re-run on the same state after a harness edit interrupted the original run. Step 14 needs a classic PAT of an organization owner, and the test account had only an OAuth token. The engine raised the expected alert.

8.2 Unit tests

42 unit tests pass, and 6 more run only against PostgreSQL. They cover number parity and mapping, webhook signatures (including refusal when no secret is set) and de-duplication, the parity report rendering, saving target-only commits before a ref is pruned, archive versioning, the deletion guard, GitLab retry and recovery after ambiguous failures, the redirector, the setup wizard (target lines, SSH quoting, the GitHub App manifest flow with a simulated GitHub, an unattended run whose output is loaded by the engine's own config loader) and the migration archive policy.

8.3 From-scratch setup test

The setup path was tested as a new user would run it, on a fresh GitLab CE 19.5.3 standing in for an Omnibus host, with its own sshd and sudo, reached over real SSH on a non-standard port:

Step	Measured	Result
`prepare-host.sh`: user, key, sudo rule, admin bot and token, 344-character target line	3 min 44 s	pass
Second run of `prepare-host.sh` (idempotent: no duplicate key or user)	≈ 4 min	pass
`./setup.sh` unattended: image, config, `check` (side channel ok over SSH + sudo), service up, first copy queued	5 min 25 s	pass
First copy of the test repository, then `verify` → `drift_count: 0`	220 s (incl. Rails boot)	pass
Interactive `ghmirror init` in a real terminal, including a bad paste	—	pass
Re-running `setup.sh` over an existing install (secrets and targets kept)	—	pass

The GitHub App is created through GitHub's manifest flow, which needs an organization owner's clicks in a browser. Everything on the engine's side of those clicks is unit-tested, but the clicks themselves were not part of the automated tests. The live tests used token authentication.

8.4 Defects found and fixed during testing

Each one is recorded in `ROADBLOCKS.md` with its cause and fix, and each is fixed and verified.

ID	Finding
E11	A create that times out after GitLab has already written could be re-sent and duplicate the object. Every create now finds and adopts the existing object first.
E12	GitLab refuses push mirrors to private-network hosts, which is where an offline secondary lives. Promote adds the secondary's host to the outbound allowlist and uses its `mirror_url`.
E13	Stopping the local `ssh` / `docker exec` did not stop the Rails agent on the GitLab host. Each agent now carries a token, is killed on the host, and exits by itself when idle.
E14	Two GitLabs on one 8 GB host overload: sizing guidance (§5.1), side-channel boot wait raised to 45 minutes.
E15–E16	Push mirrors run at most every 5 minutes; GitLab's branch cache lags. Both are documented and handled.
E17	`verify` now also reports objects that exist only on GitLab.
E18–E19	Promote interrupted midway: `demote` restores everything. An unreachable secondary at promote is now a reported failure, not a silent skip.
E20–E22	Side channel over real SSH: argument quoting, `pkill` self-match, sudo path matching.
E23–E24	Compose stack: mounts, volume ownership, secret permissions, port clash, database start-up race.
E25	GitLab 19's new name-clash message; personal-account group naming.
E26–E28	Paste-length limit of terminals; a false drift alert on day one; weekly migration alert noise with GitHub Apps.

E29	A branch made by hand on a mirror was removed by the next ref-exact push. Commits that exist only on GitLab are now saved in the engine's git mirror (`refs/ghmirror-backup/<time>/target/`) before any ref is deleted or overwritten. Verified live (chapter 14).
E30	Every push re-versioned every PR in the L2 archive (timestamps of the embedded repository changed). Volatile fields of embedded repositories no longer count as a change.

9 Limits and known gaps

Area	Limit
Not exercised end to end	Organization teams and members, GitHub App authentication against live GitHub, wikis and LFS are implemented and unit-tested, but the live tests used a personal account and a token.
No GitLab equivalent	GitHub Discussions, Projects (boards), Actions run history and logs: not copied and not archived.
Never available	Secret values (Actions secrets, private halves of deploy keys): GitHub never exposes them. Re-create them in GitLab.
Side channel needed	MR creation dates, squash-merged state and approval dates need the side channel. Without it they become notes.
Restore from L2	The JSON archive is not replayed into GitLab automatically. Rebuild from GitHub or from the other GitLab (§7.5).
After failover	The secondary receives git data only, and up to 5 minutes late. The engine no longer syncs metadata between the GitLabs.
Usernames	A mirror account takes the GitHub login. If a GitLab user or group already has that name, it becomes `<login>-gh` and the engine log records it. Check Admin › Users after the first copy.
Hosting	Self-managed GitLab only. GitLab.com cannot impersonate users or set historical dates. Tested on 19.5.3; re-run `check` and `verify` after upgrades.
Security	The side channel is root-equivalent on each GitLab host (§5.4). Protect the engine host accordingly.

10 Troubleshooting

Symptom	Likely cause	Fix
`side channel FAILED … a password is required`	The sudo rule does not name the path sudo resolves.	Re-run the current `prepare-host.sh`, which lists every path.
`Host key verification failed`	The host was re-installed, or the engine uses another name for it.	Re-run `prepare-host.sh` (with `--ssh-host`) and paste its line into `./setup.sh`.
`agent not ready after 2700s`	The GitLab host is overloaded; Rails does not boot.	Check memory and `gitlab-ctl status`; size the host per §5.1.
`target …: UNAVAILABLE` with 401	Bot token expired or revoked.	Re-run `prepare-host.sh` and `./setup.sh`.
Paste prompt does not accept the line	Line cut off while copying.	Copy the whole line, or pass it as `--target NAME=ghmirror-target:…`.
Browser cannot open `localhost:8765`	The engine runs on a remote server.	`ssh -L 8765:localhost:8765 <engine-host>`, then reload.
Webhook deliveries fail with 401	The secret differs between GitHub and `.env`.	Re-run `./setup.sh` (App: `--new-app`), or copy the secret from `.env` into the webhook settings.
Changes appear only every 15 minutes	No public webhook URL; polling mode.	Expected. For seconds, set up the webhook path (§5.5).
`ghmirror_halted 1`	Deletion guard.	§7.2.
Dead jobs	A job failed 5 times.	`docker compose logs engine` for the error, fix it, then `enqueue-all`.
Secondary not updated right after promote	GitLab's 5-minute push-mirror schedule.	Wait 5 minutes; see Settings › Repository › Mirroring repositories.
Migration archive alert	No classic PAT for the migrations API.	Optional. Set `GITHUB_MIGRATION_TOKEN` in `.env` (owner, `repo, admin:org`).
setup: files appear empty in the container (macOS)	Docker Desktop or colima shares only some folders (colima: the home folder).	Keep the repository under your home folder.

11 Security architecture and threat model

The engine host holds the keys to both GitLabs and a full copy of the code, so it must be protected like the GitLab servers themselves. This chapter summarises `docs/SECURITY.md`. Every statement there was checked against the code.

11.1 Trust boundaries

Boundary	Authentication	Protection
GitHub → engine (webhooks)	HMAC-SHA256 over the raw body, constant-time comparison. With no secret configured, every webhook is refused (fail closed). Bodies over 26 MB are refused; duplicate deliveries are dropped.	The engine listens on `127.0.0.1:8080` behind your proxy. A webhook only queues a job, and the job re-reads the object from GitHub, so webhook content is never trusted.
Engine → GitHub	GitHub App installation token (1-hour lifetime) or a token.	The App is **read-only**: contents, metadata, issues, pull requests, discussions, administration, members. It cannot write to GitHub.
Engine → GitLab API	Admin bot token (`api sudo admin_mode read_api write_repository`), 365-day lifetime by default.	HTTPS. One token per GitLab, so a leak does not cross to the other GitLab. `check` reports the days left.
Engine → GitLab host (side channel)	SSH key only, with the host key pinned at setup (`StrictHostKeyChecking=yes`).	The login has no pty and no forwarding. The agent runs a fixed set of operations.
People → GitLab	GitLab's login, SSO, 2FA.	While mirroring, mirror accounts are blocked and people have read access only.

Git credentials travel in `http.<url>.extraheader` through `GIT_CONFIG_*` environment variables. They never appear in a remote URL, in `ps` output or in `.git/config`.

11.2 Threats and mitigations

Threat	Impact	Mitigation
Engine host compromised	**Highest:** admin of both GitLabs, root on both GitLab hosts, read access to every repository, the full copy on disk.	Dedicated VM, disk encryption, no inbound ports except `/webhook` , operator SSH with keys and 2FA, secrets in 0600 files, encrypted backups of `.env` and `secrets/` .
Side-channel key stolen	Root-equivalent on that GitLab host: `gitlab-rails` or `docker exec` can run any code.	Key unique to ghmirror; pinned host key; `from="<engine IP>"` in `authorized_keys` or a firewall rule; audit `ghmirror` logins. If unacceptable, run without the side channel (§9).
Bot token leaked	Admin API on one GitLab.	Limited lifetime; revoke in the admin area; audit events (EE).
Forged webhooks	Without a secret: work queued (denial of service). No data change.	HMAC with a 256-bit secret, fail closed, size cap, de-duplication.

Mass deletion on GitHub (stolen admin, ransomware, mistake)	Copies destroyed, if mirrored blindly.	ghmirror never deletes issues, merge requests or projects. Deleted repositories are archived. The deletion guard halts all writes on a mass disappearance. Rewritten git refs keep their old tips in L1, and commits that exist only on GitLab are saved before a ref is reset (E29).
A GitLab instance compromised	That copy read or altered.	Data flows only from GitHub. The nightly `verify` detects changes and `--repair` restores them.
`/status`, `/metrics` exposed	Repository names and queue state revealed.	Both are bound to localhost. Expose only `POST /webhook`.

11.3 Hardening checklist

Before going live

- Dedicated, encrypted VM.
- Only `POST /webhook` exposed, over HTTPS.
- A GitHub App rather than a personal token; `GITHUB_MIGRATION_TOKEN` empty unless needed.
- GitLab URLs on `https://`.
- `from=` restriction on the side-channel key.
- Token rotation reminder; alerts to a watched channel.
- Backups of secrets kept apart from the data.
- A failover drill twice a year.

Known limitations:

- The side channel is root-equivalent by design.
- Volumes are not encrypted by ghmirror itself.
- `/status` and `/metrics` are unauthenticated.
- Rotation is manual.
- There has been no external audit yet.

12 Capacity, API limits and recovery objectives

Figures marked **measured** come from the drill in chapter 14: one repository with 2 issues, 5 PRs, 20 comments and reviews, 3 accounts and a release, on GitLab CE 19.5.3 sharing a 4-CPU / 8 GB VM with the engine. Figures marked estimate scale those measurements up. The full method is in `docs/CAPACITY.md`.

12.1 Storage

Store	Size	Basis
L1 git mirrors (engine)	≈ the git size of the mirrored repositories, plus LFS	bare mirrors
L2 archive (engine)	≈ 35 KB per PR, 15 KB per issue, plus attachments and release assets at full size, plus one copy per real change	**measured**: PR 32–51 KB, issue 13–15 KB
State database (engine)	≈ 1–2 KB per object	**measured**: 64 KB for about 40 objects
Each GitLab	git size + attachments (uploads) + release assets (packages) + GitLab's own database	same objects as GitHub

Engine disk: plan ≥ 1.2 × (git + LFS) + 50 KB × (issues + PRs) + attachments + assets + 20 GB.

Example (estimate): 300 repositories, 40 GB git, 5 GB LFS, 35,000 issues and PRs, 10 GB of attachments. That needs about 100 GB on the engine and about 70 GB per GitLab, plus GitLab's baseline.

12.2 GitHub API limits

Authentication	Requests per hour (GitHub documentation, checked 2026-11-04)
GitHub App installation	5,000 minimum. +50 per repository above 20 and +50 per member above 20; at most 12,500.
GitHub App on Enterprise Cloud	15,000
Personal access token / OAuth token	5,000, shared with everything else that user does

Operation	API cost
First copy	**measured** 32 REST calls and 0 GraphQL for about 30 objects: about 1 call per object
Nightly parity check	**measured** 16 calls for 1 repository with 7 issues and PRs: about 4 per repository + 2 per PR
At the limit	The engine sleeps until the reset (`Retry-After` / `X-RateLimit-Reset`) and continues; nothing is skipped. `git fetch` does not count against the limit.

Example (estimate): a first copy of 35,000 issues and PRs needs about 40,000 calls, which is 3.5 hours of a 12,500/h App budget. A nightly verify needs about 41,000 calls. On a 5,000/h token, run verify weekly.

12.3 Time for the first copy

Measured: 168 s in total.

- Group and project: 5 s.
- Mirror accounts: 5–20 s each (avatar and keys).
- Side-channel Rails boot: 25–60 s, once per engine start.
- Issues and merge requests with their comments: about 60 s, or about 2 s per issue or PR.

Estimate: 35,000 issues and PRs over 4 workers take about 5 hours plus the git transfer, so plan a weekend. The copy is resumable.

12.4 Recovery objectives

Objective	Value	Basis
RPO, with webhooks	**2–5 s** (push), **8–9 s** (comment)	**measured**: GitHub change to GitLab, 5 probes
RPO, polling only (no public endpoint)	≤ 15 min	`schedule.git` / `meta` = 900 s
Engine step of the RTO: `promote`	**7.8–8.3 s**	**measured**: 1 project, 1 account
… for 300 projects and 200 people	≈ 7 min	estimate, linear in API calls
… with a reachable secondary	+ 65 s once	GitLab caches the outbound allowlist
Secondary lag after promote	≤ 5 min	GitLab push-mirror interval (e2e step 10)
`demote`	**1.2–2 s**	**measured**
Expected RTO, from decision to developers pushing	15–30 min	dominated by communication; see the runbook (§7.3)

13 Supported platforms and offline-mode limits

The status words are: tested exercised end to end; built implemented and unit-tested; expected same API as a tested platform but untested; unsupported. Full table: `docs/SUPPORT-MATRIX.md`.

13.1 Editions

Platform	Status	Notes
GitHub.com, personal account	tested	e2e, setup test and the drill
GitHub.com organization (Free, Team)	built	unit-tested; no organization in the e2e
GitHub Enterprise Cloud (incl. EMU, data residency)	expected	data residency: set `api_url` / `web_url` by hand
GitHub Enterprise Server 3.x	expected	`api_url: https://HOST/api/v3`; token mode in `init`
GitLab CE 19.5.3, Omnibus (Docker or package)	tested	e2e with two instances, setup test over SSH, the drill
GitLab CE/EE 19.x self-managed	expected	re-run the e2e before upgrading GitLab across a minor version
GitLab older than 19.0	not tested	likely back to 17.x; unsupported until tested
GitLab Helm chart (Kubernetes)	not built	would need a `kubectl exec` side channel
GitLab.com, GitLab Dedicated	unsupported	no impersonation or historical dates; no host access

13.2 Exact limits of the offline (on-premises) copy

Limit	Detail
Not air-gapped	The engine pushes to it over the network and must reach GitHub at the same time. There is no export-to-media mode.
Read-only while mirroring	Changes made by hand are reported by `verify` and may be overwritten from GitHub. Commits that exist only on GitLab are saved first (E29).
Freshness	Seconds with webhooks, up to 15 minutes when polling; catches up after engine downtime.
After failover	The secondary receives git only, within 5 minutes. Issues and merge requests created after the failover are not copied to it.
Links in text	Attachments, avatars and release assets are copied; links in text still point at GitHub.
CI	Disabled while mirroring, enabled by `promote`. Runners and variables must be prepared on the offline GitLab.
One engine	One engine feeds both GitLabs. If it is lost, updates stop, but the copies stay intact.
Restore from L2	Manual (scripted). The fast path is a new first copy from GitHub, or export/import from the surviving GitLab.

14 Recorded failover drill and parity report

This drill was recorded live on 2026-11-04. A scratch GitHub repository was mirrored into GitLab CE 19.5.3, GitLab was promoted, a developer shipped a fix on GitLab, and the drill was ended with demote. The video (3 min 16 s), the raw terminal recordings and a transcript are in `docs/demo/`.

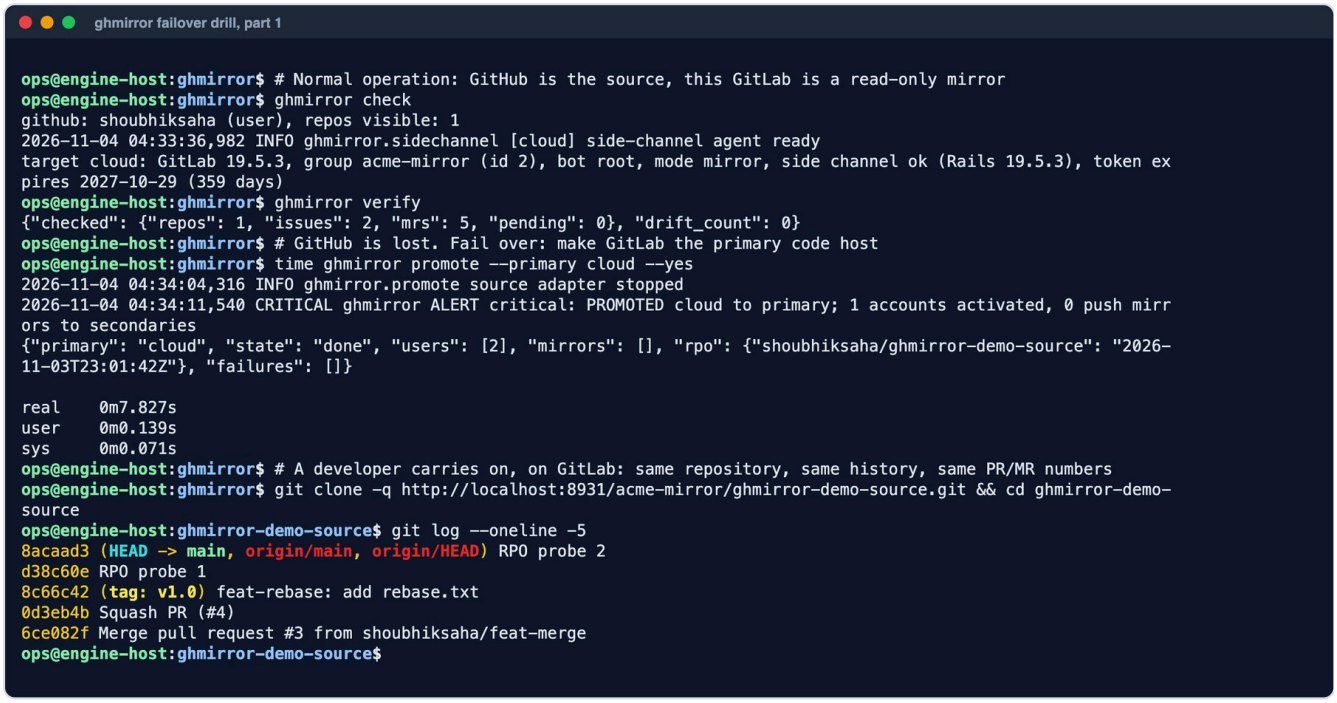


Figure 25. A healthy mirror (`check`, `verify` with 0 differences), then `promote` in **7.8 s** with `failures: []` and the last sync time per repository (`rpo`). The developer then clones from GitLab: same history, same tags.

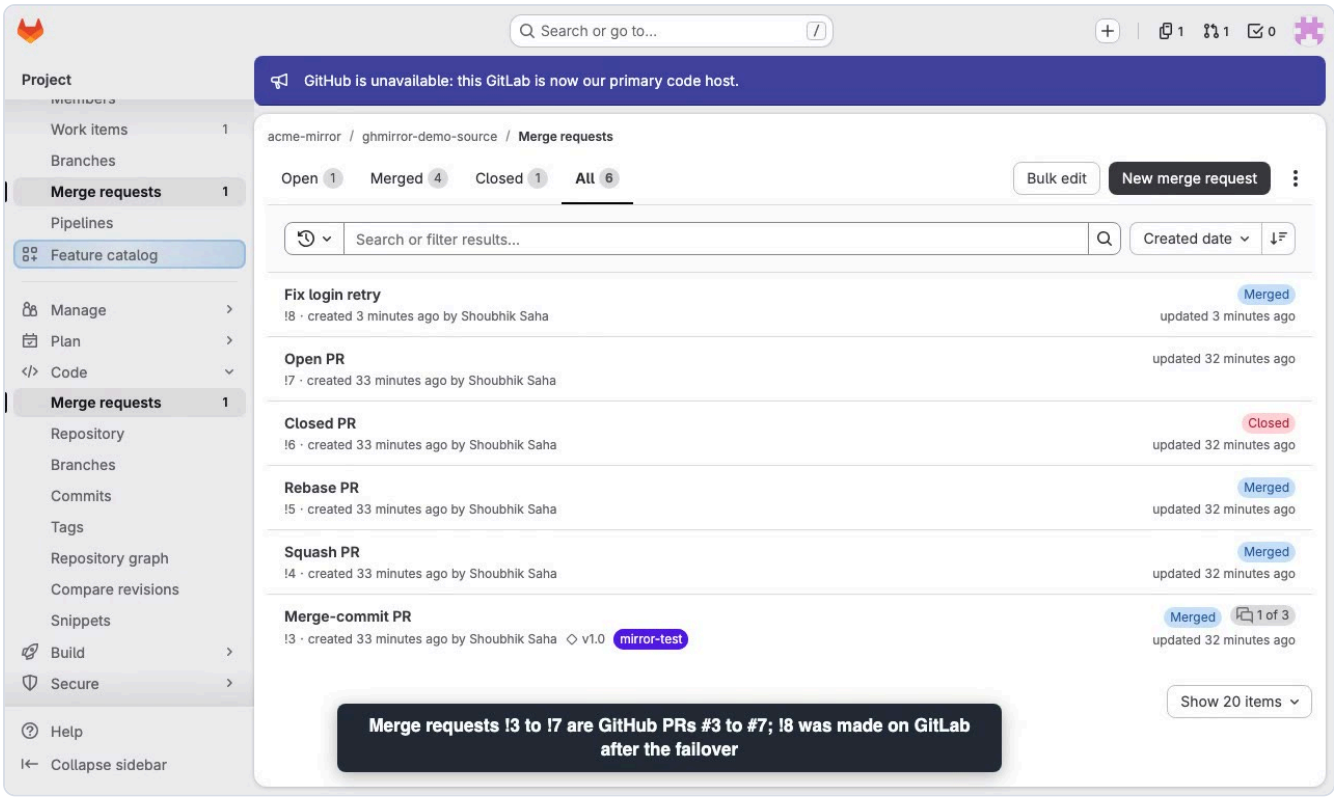


Figure 26. The developer's view, signed in with their own account after promote. The failover banner is on every page. Merge requests !3 to !7 are GitHub's PRs #3 to #7. !8 was pushed, opened and merged on GitLab during the outage.

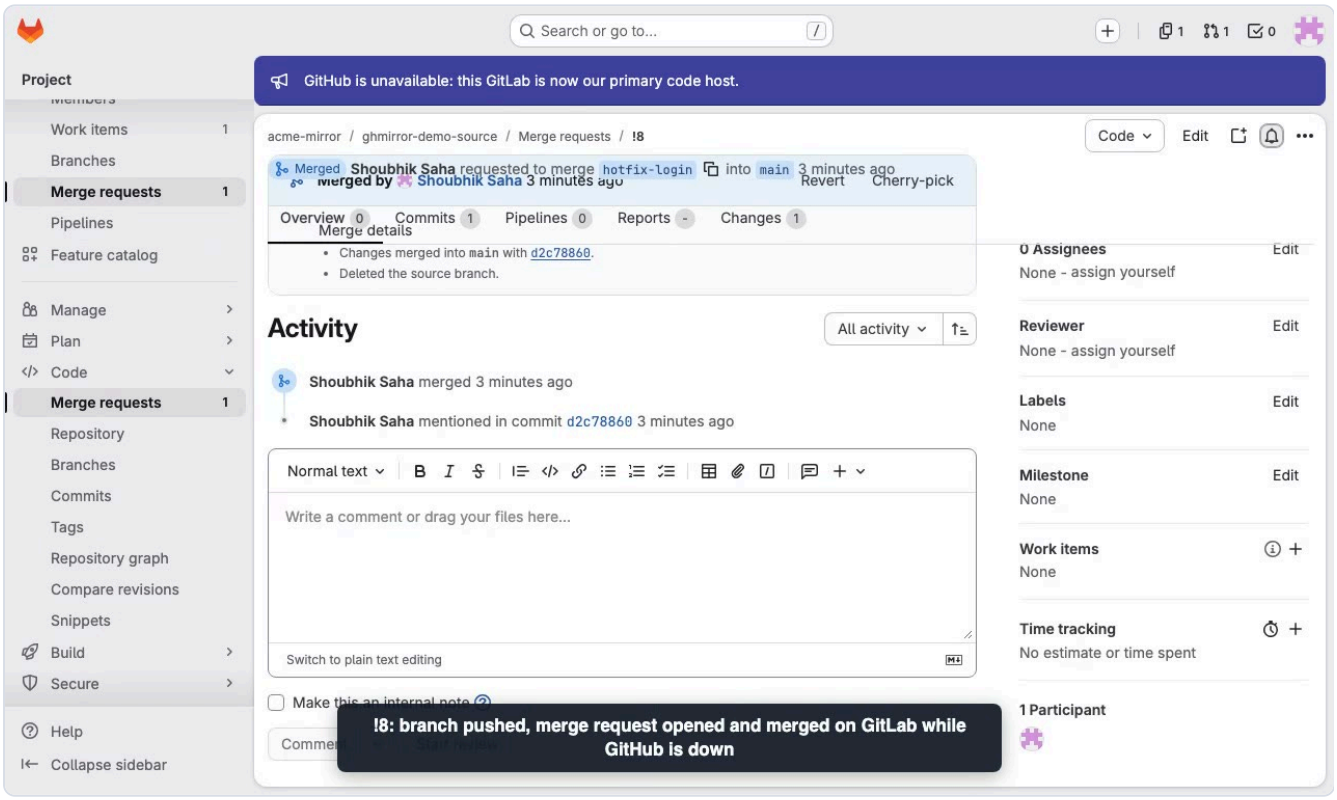


Figure 27. !8, merged by the developer's own account while GitHub was down.



Figure 28. `demote` in **1.2 s**. The parity check then reports the drill's work instead of losing it: `main` moved on GitLab, and !8 exists only on GitLab.

14.1 Sample parity report

`ghmirror verify --format markdown` renders the nightly check for people. The reports below are real; the JSON versions and full texts are in `docs/samples/`. After the first copy the result was `PASS`: 1 repository, 2 issues and 5 PRs compared, 0 differences. Four changes were then made by hand on the GitLab mirror. The next report found all four:

Object	Field	GitHub	GitLab
`refs/heads/local-experiment`	sha	*(none)*	`8c66c426…`
issue #1	title	Crash on startup	Crash on startup (edited on the mirror)
issue #1	labels	bug, mirror-test	mirror-test
issue #3	exists	absent	present

After `ghmirror verify --repair` and the queued re-sync, the title, the label and the branch matched GitHub again. The branch was removed only after its commit had been saved in the engine's git mirror. Issue #3, which was made by hand, remained in the report: ghmirror never deletes issues, so a person decides.

15 Commercial deployment and support (draft)

This chapter is a draft. Price, licence, legal entity and response times are the owner's decisions; the full draft is `docs/COMMERCIAL.md`.

15.1 Offering

Element	Content
Packaging (to decide)	Commercial EULA, source-available (recommended, so buyers can audit a tool with root-equivalent access), or open source with paid services.
Deployment service	1. Assessment: sizing, network, identity. 2. GitLab hosts. 3. Engine install and hardening. 4. First copy, ending in a parity report that passes. 5. Failover drill and handover. About 8–15 consultant days.
Support tiers	• **Standard:** business hours. • **Business:** two time zones, one assisted drill a year. • **Critical:** 24×7 for Severity 1, two drills a year, a named engineer during a real failover.
Severity 1 target (proposal)	Failover needed and `promote` fails, or the mirror is halted while GitHub is down. Response: 1 business day (Standard), 4 hours (Business), 1 hour 24×7 (Critical).
Release policy (proposal)	Semantic versions with tagged releases; the current and previous minor release supported; each monthly GitLab release run through the e2e within 10 working days.

15.2 Readiness gaps before the first paying customer

#	Gap	Why it matters
1	No e2e with a GitHub organization (teams, App clicks)	Organizations are the main customers.
2	No load test on a realistic estate	Chapter 12 extrapolates from one repository.
3	No CI pipeline, tagged releases or signed image	Needed for the compatibility promise.
4	No versioned schema migrations	Upgrades that change the state schema need a path.
5	No external security review	The engine holds high-value credentials.
6	No LICENSE / EULA; product name not checked	Cannot be sold or evaluated legally without them.
7	Single maintainer; no GitLab-to-GitLab metadata sync after failover	Support cover; completeness of the secondary.

A Command reference

Run `ghmirror -c ghmirror.yaml <command>`, or `docker compose run --rm engine <command>` in `deploy/compose/`. The config path is preset in the container.

Command	Purpose
`init`	Guided setup: writes `ghmirror.yaml`, `.env`, `identities.yaml`, `secrets/`. Flags: `--dir --owner --owner-type --github-auth app|token --github-token --new-app --webhook-url --target-names --target NAME=LINE --group --include --alert-webhook --callback-port --callback-bind --yes`.
`check`	GitHub access, admin tokens, groups (created if missing), side channels. Exit 1 on any failure.
`run`	The service: webhook receiver on `/webhook`, `/healthz`, `/metrics`, `/status`, queue workers, scheduler.
`sync [--repo R] [--full] [--org] [--no-git] [--no-meta]`	One-off sync without the queue. Exit 3 if the guard halts.
`enqueue-all`	Queue a full pass for every selected repository (the service works through it).
`verify [--repo R] [--repair] [--format json|markdown] [--output F]`	Parity report. Exit 1 on drift. `--repair` re-syncs drifted objects. `--format markdown` prints the readable report (chapter 14); `--output` also writes it to a file.
`status`	Mode, queue, last parity report, recent alerts (JSON).
`guard status|reset`	Show or clear the deletion-guard halt.
`promote --primary NAME [--yes]`	Fail over to a GitLab target. Exit 1 if any step failed (listed).
`demote`	Undo promote exactly, from its record (drills).
`redirector [--listen ADDR]`	Serve old GitHub URLs as redirects to the primary GitLab.
`archive-migration [--no-wait]`	Run GitHub's migration export now and store it in the archive.

Script	Purpose
`deploy/compose/setup.sh`	Build, `init`, `check`, start, first copy. Extra arguments go to `init`.
`prepare-host.sh`	Run on a GitLab host as root: `--pubkey --container --ssh-host --ssh-port --bot --token-days`.
`e2e/run_e2e.sh`	The end-to-end test (needs Docker and a GitHub token; creates and deletes its own repositories).

B Configuration reference

`ghmirror.yaml`. Any value can be `${VAR}` or `${VAR:-default}`, taken from the environment (`.env`). Defaults are shown.

Key	Meaning
`source.owner`, `owner_type` (`org`)	GitHub organization or user, and its type.
`source.api_url`, `web_url`	For GitHub Enterprise Server (default github.com).
`source.auth.app_id`, `app_private_key_file`, `app_installation_id`	GitHub App authentication (or `app_private_key` inline).
`source.auth.token`	…or a classic PAT.
`source.auth.migration_token`	Optional classic PAT (`repo, admin:org`) for the weekly migration archive.
`source.webhook_secret`	HMAC secret for webhook signatures.
`source.repos.include` (`["*"]`), `exclude`	Glob patterns on repository names.
`targets[].name`, `url`, `token`, `group`	One entry per GitLab: name, base URL, admin bot token, top-level group.
`targets[].side_channel`	Command prefix that runs a program on the GitLab host (`ssh … sudo`, or `docker exec -i NAME`). Omit to disable.
`targets[].visibility_cap` (`private`)	Most open visibility a mirrored project may get.
`targets[].mirror_url`, `git_url`	How the other GitLab reaches this one (push mirrors after promote); a git URL override.
`targets[].ci_config_path`	Optional external CI configuration path set on each project, for example `pipelines/{repo}.gitlab-ci.yml@{group}/ci-config`.
`state.database`	`postgresql://…` (compose) or `sqlite:///…`.
`state.mirrors_dir`, `archive_dir`	L1 bare mirrors and L2 archive.
`identity.email_domain`	Domain for placeholder emails of people missing from the map.
`identity.email_map_file`, `email_map`	GitHub login → company email (file next to the config, or inline).
`identity.import_keys`, `import_avatars` (`true`)	Copy SSH/GPG keys and avatars to mirror accounts.
`sync.mirror_attachments` (`true`)	Copy files pasted into issues and comments.
`guard.max_disappear_ratio` (0.02), `max_disappear_count` (1)	Halt when more than this share *and* count vanish in one pass.
`schedule.repos/meta/git/org/full/verify/migration`	Seconds: 3600 / 3600 / 21600 / 21600 / 86400 / 86400 / 604800.
`alerts.webhook_url`	Slack-compatible incoming webhook.
`server.listen` (`0.0.0.0:8080`), `workers`	Service address and worker threads.

`redirect.target`, `base_url`	Redirector destination target and public base URL.
`promote.broadcast_message`	Banner text shown after promote.

C Repository layout

Path	Content
`README.md`	Start here: what it is, quick start
`DESIGN.md`	Full design and decisions
`RESEARCH-prior-art.md`	Existing tools and what was taken from them
`ROADBLOCKS.md`	Every risk checked and every defect found (E1–E30)
`docs/RUNBOOK.md`	Operations, short form of chapter 7
`docs/INSTALL.md`, `examples/`	Reproducible installation; sample configuration
`docs/SECURITY.md`, `CAPACITY.md`, `SUPPORT-MATRIX.md`	Chapters 11, 12 and 13 in full
`docs/demo/`, `docs/samples/`	Recorded failover drill; sample parity reports (chapter 14)
`docs/COMMERCIAL.md`	Draft commercial offering (chapter 15)
`docs/ghmirror-guide.pdf`, `docs/guide-src/`	This guide and its sources
`engine/ghmirror/cli.py`	Commands (appendix A)
`engine/ghmirror/setup.py, prepare-host.sh`	`ghmirror init`; GitLab host preparation
`engine/ghmirror/engine.py`	Targets, deletion guard, alerts
`engine/ghmirror/sync.py`	Repositories, issues, PRs, reviews, releases
`engine/ghmirror/identity.py, org.py`	Mirror accounts; members and teams
`engine/ghmirror/promote.py, verify.py`	Promote/demote; parity report
`engine/ghmirror/sidechannel.py, .rb`	The Rails agent and its driver
`engine/ghmirror/server.py`	Webhooks, queue workers, scheduler, metrics
`engine/ghmirror/gitmirror.py, archive.py, migration.py`	L1 git mirrors; L2 archive; migration archive
`engine/ghmirror/github.py, gitlab.py, db.py, mapping.py`	API clients, state database, field mapping
`engine/tests/`	Unit tests
`deploy/compose/`	`setup.sh`, `docker-compose.yml`, `Dockerfile`, `entrypoint.sh`, example config
`deploy/gitlab/`	GitLab CE template
`e2e/`	End-to-end test harness
`verification/`	The roadblock test scripts and their recorded results